VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Network Security

Assignment Report

Network Security Monitoring and Security Analytics

Advisor: Nguyen Duc Thai

Student: Nguyễn Thanh Thảo 2570758
Nguyễn Đình Phú 2470891
Bùi Bá Phúc 2570799

HO CHI MINH CITY, NOVEMBER 2026

Contents

1 Introduction and Project Objectives **6**

2 Network Security Monitoring Fundamentals **8**
2.1 Overview 8
2.2 Core Concepts 8
2.2.1 Network Traffic Monitoring 8
2.2.2 Packet Capture (PCAP) 8
2.2.3 Network Flows 9
2.2.4 Metadata 9
2.2.5 Logs 9
2.2.6 Security Events 10
2.2.7 Indicators of Compromise (IoC) 10
2.2.8 Indicators of Attack (IoA) 10
2.2.9 Security Alerts 10
2.2.10 Security Analytics 10

3 Network Data Sources **12**
3.1 Packet Captures (PCAP) 12
3.1.1 Data Structure 12
3.1.2 Identifiable Security Events 12
3.1.3 Advantages and Limitations 13
3.1.4 Privacy and Compliance Concerns 13
3.2 DNS Traffic 14
3.2.1 Data Structure 14
3.2.2 Identifiable Security Events 14
3.2.3 Advantages and Limitations 14
3.2.4 Privacy and Compliance Concerns 14
3.3 HTTP and HTTPS Traffic 15
3.3.1 Data Structure 15
3.3.2 Identifiable Security Events 15
3.3.3 Advantages and Limitations 16
3.3.4 Privacy and Compliance Concerns 16

4 Security Analytics and Detection **17**
4.1 Detection Approaches . . . 17
4.2 Known Indicators vs. Unknown or Previously Unseen Behaviour . . . 18
4.3 False Positives and False Negatives . . . 19
4.4 From Alert to Decision . . . 20

5 Related Work / Case Study **21**
5.1 Literature Review: Theoretical Foundations of Network Security Monitoring 21
5.2 Case Study 1: The Equifax Breach (2017) — Visibility Constraints in Encrypted Traffic . . . 21
5.3 Case Study 2: The Target Corporation Breach (2013) — Alert Fatigue and the NSM Lifecycle . . . 22

6 Laboratory Environment **23**
6.1 Generated-Traffic Lab . . . 23
6.2 Real-Traffic Lab . . . 23
6.3 Safety Controls . . . 23

7 Experimental Methodology **25**
7.1 Packet Analysis . . . 25
7.1.1 Objectives and Experimental Phases . . . 25
7.1.2 Traffic Generation and Benign Probing . . . 25
7.1.3 Packet Collection and Analysis . . . 25
7.2 Network Flow Analysis . . . 26
7.2.1 Data and Method . . . 26
7.2.2 Flow Characteristics and Suspicious Pairs . . . 28
7.2.3 Beacon Detection and Evaluation . . . 29
7.3 Security Dashboard Analysis . . . 32
7.3.1 Objective and Data . . . 32
7.3.2 Pipeline . . . 32
7.3.3 Dashboard Design . . . 33
7.3.4 Analysis and Rule Tuning . . . 34

8 Network Traffic Log Analysis **35**
8.1 HTTP Flooding . . . 35
8.1.1 Observed Pattern . . . 35
8.1.2 Response Timing and Service Impact . . . 35

8.1.3 Connection Closure and Assessment . . . 35
8.2 TCP SYN Flooding . . . 36
8.2.1 Observed Pattern . . . 36
8.2.2 Incomplete Handshakes . . . 36
8.2.3 Effect on Legitimate Connections . . . 36
8.2.4 Late Resets and Continuing Retransmissions . . . 37
8.3 Command and Control Activities . . . 38
8.3.1 Observed HTTP Activity . . . 38
8.3.2 Analysis of HTTP Behaviour . . . 39
8.3.3 Detection Significance . . . 39
8.3.4 Payload and Infrastructure Behind the Channel . . . 40
8.4 Indicators of Compromise (IoCs) . . . 41
8.4.1 HTTP Flood Indicators . . . 41
8.4.2 SYN Flood Indicators . . . 41
8.4.3 Malware . . . 41
8.4.4 Indicators Extracted from This Capture . . . 42
8.4.5 Limits of Indicator-Based Detection . . . 43

9 Security Investigation Scenario 44
9.1 Scenario Setup and Initial Alert . . . 44
9.2 Evidence Collection and Investigation . . . 45
9.2.1 Host Identification and Traffic Pivoting . . . 45
9.2.2 Reconstruction of Suspicious Activities . . . 46
9.2.3 Payload and Communication Analysis . . . 46
9.2.4 Validation and Differential Diagnosis . . . 48
9.2.5 Detection Gap and Remaining Uncertainty . . . 48
9.3 Investigation Conclusion and Response . . . 49

10 Experimental Results 51
10.1 HTTP and SYN Flooding . . . 51
10.2 Dashboard Monitoring Results . . . 51
10.3 From Indicators to Detection Rules . . . 55
10.4 Rule Set Evaluation . . . 56
10.4.1 Alert Volume . . . 56
10.4.2 Alert Verdicts . . . 56
10.4.3 Coverage by Attack Stage . . . 57

10.4.4 False Negatives and False Positives 57

11 Security Analysis and Discussion **59**
11.1 Distinguishing Normal from Suspicious Behaviour 59
11.2 Correlating Multiple Pieces of Evidence 60
11.3 How Security Analytics Identified Suspicious Behaviour 61
11.4 Limitations 61

12 Proposed Monitoring Architecture **63**
12.1 Scope and Goal 63
12.1.1 Hypothetical Organisation 63
12.1.2 Goal 63
12.1.3 Scope 64
12.2 Architecture Diagram 64
12.3 Analysis: Flows, Packets and Signatures Together 66
12.4 Information Flow and Process 66

13 Conclusion **68**

14 References **69**

A Laboratory Details **71**
A.1 Generated-Traffic Lab 71
A.2 Real-Traffic Lab 71

1 Introduction and Project Objectives

Background. Organisations collect large volumes of network traffic and logs, but this data is only useful when it becomes reliable security evidence. Network Security Monitoring (NSM) is the practice of collecting, analysing and escalating that evidence so that suspicious activity can be detected, investigated and answered.

Problem statement. This project asks how network traffic and security data can be analysed to identify suspicious activity and support incident detection. It also asks where indicator-based detection stops working and behavioural analysis has to take over.

Project objectives. Over six weeks, the group aimed to:

1. explain the core NSM concepts and evaluate three data sources: packet captures, DNS traffic and HTTP/HTTPS traffic;
2. compare detection approaches and their false-positive and false-negative trade-offs;
3. carry out two practical investigations, packet analysis (Option A) and network flow analysis (Option B);
4. identify security-relevant activity in the traffic, extract indicators and assess their limits;
5. follow one alert through a full investigation, from first signal to a reasoned conclusion and response;
6. propose a monitoring architecture for a small organisation.

Project scope. The practical work used two data sets:

- **A controlled lab.** We built a network with four containers: a target, an attacker, a benign probe and a packet sniffer. We generated an HTTP flood and a TCP SYN flood against the target, captured the traffic with tcpdump, and compared it with the probe's results, resource measurements and nginx logs.
- **A public malware capture.** This is the 2025-01-22 malware-traffic-analysis exercise, a 53-minute capture of a Windows domain workstation infected through a fake Google Authenticator download. We analysed it at flow level and request level, without replaying traffic or running any payload.

Across both data sets we identified three security-relevant activities: HTTP flooding, SYN flooding, and command-and-control polling over HTTP. The malware capture also

served as the basis for the investigation scenario. Live production networks, endpoint telemetry and TLS decryption were out of scope.

2 Network Security Monitoring Fundamentals

2.1 Overview

Network Security Monitoring (NSM) is the practice of collecting, analyzing, and escalating network-based evidence so that analysts can detect and investigate suspicious activity. Its value does not come from storing packets and logs. It comes from turning that data into reliable security evidence that supports detection, investigation, and incident response.

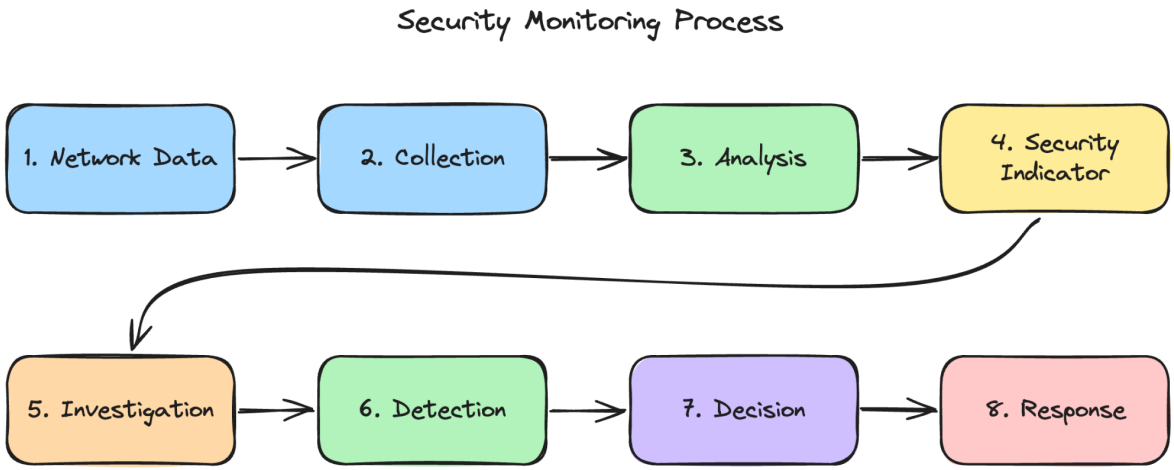


Figure 2.1: The security monitoring process.

Figure 2.1 shows the security monitoring process.

2.2 Core Concepts

2.2.1 Network Traffic Monitoring

Network traffic monitoring is the observation of traffic crossing a network. It can serve operations (capacity, availability) or security (detecting threats). Traffic is usually observed at a choke point such as a switch SPAN port, a network TAP, or a virtual switch in a lab. In this project it is used for security purposes: identifying who talks to whom, how often, over which protocols, and whether this matches expected behavior.

2.2.2 Packet Capture (PCAP)

Packet capture records individual packets, including headers and, if available, payload.



- **Provides:** source/destination IP, ports, protocol, TCP flags, DNS query names, HTTP requests, timing.
- **Strength:** the most detailed evidence; allows reconstruction of a conversation.
- **Limitations:** large storage cost; encrypted payload (TLS) is not readable; capturing user traffic raises privacy concerns.
- **Tools:** tcpdump, Wireshark.

2.2.3 Network Flows

A flow summarizes a conversation between two endpoints over a period, typically by its 5-tuple (source IP, destination IP, source port, destination port, protocol) plus start time, duration, and byte/packet counts.

- **Strength:** compact, scalable, works even when payload is encrypted.
- **Limitation:** no content, so it shows *that* communication happened, not *what* was said.
- **Example:** one source contacting 200 different ports on one host within seconds suggests scanning.

2.2.4 Metadata

Metadata is structured information extracted from traffic that describes it without storing the whole content, such as DNS queries and answers, HTTP host and user agent, TLS server name (SNI), and certificate details. Zeek is a network analysis tool that produces this kind of metadata as structured logs (e.g., conn.log, dns.log, http.log) and links them with a connection identifier (uid), which makes pivoting between logs possible. Metadata sits between flows and full packets: richer than flows, far smaller than PCAP.

2.2.5 Logs

Logs are records of events produced by systems and applications, such as firewalls, DNS servers, authentication services, IDS/IPS, network devices, and endpoints. NIST SP 800-92 describes log management as the processes of generating, transmitting, storing, analyzing, and disposing of log data. Logs show *what a system did or decided* (e.g., "login failed", "connection blocked"), which complements what the network shows. Their reliability depends on accurate timestamps, adequate retention, and protection from tampering.

2.2.6 Security Events

A security event is any observable occurrence relevant to security, whether benign or malicious: a failed login, a blocked connection, an NXDOMAIN response, a port being probed. Most events are harmless on their own. Events are the raw material from which alerts and incidents are built.

2.2.7 Indicators of Compromise (IoC)

IoCs are artifacts that suggest a system may already be compromised or has interacted with known malicious infrastructure. Network-related examples are IP addresses, domain names, URLs, file hashes, unusual ports, and suspicious DNS requests.

- **Use:** fast matching of traffic and logs against known-bad lists.
- **Weakness:** they describe what was already seen; attackers can change IPs, domains, and hashes cheaply, so IoC matching misses new variants.

2.2.8 Indicators of Attack (IoA)

IoAs describe the *behavior or technique* of an attack in progress, regardless of the specific infrastructure used. Examples are port scanning, repeated authentication failures, periodic beacon-like connections, and bursts of failed DNS lookups.

- **Use:** detecting unknown or modified attacks by their behavior.
- **Weakness:** more false positives, because legitimate tools can behave similarly (e.g., an authorized vulnerability scanner).

2.2.9 Security Alerts

An alert is a notification generated when data matches a signature, rule, threshold, or analytic. An alert is a *hypothesis that needs triage*, not a confirmed incident. Poorly tuned detection produces false positives and alert fatigue; missing coverage produces false negatives (discussed in Section 11).

2.2.10 Security Analytics

Security analytics applies methods such as rules, thresholds, statistical analysis, baselining, anomaly detection, behavioral analysis, correlation, and threat intelligence to security data in order to find suspicious behavior and support decisions. Correlation is especially

important: combining DNS, flow, packet, and authentication evidence gives higher confidence than any single source (detailed in Section 4).

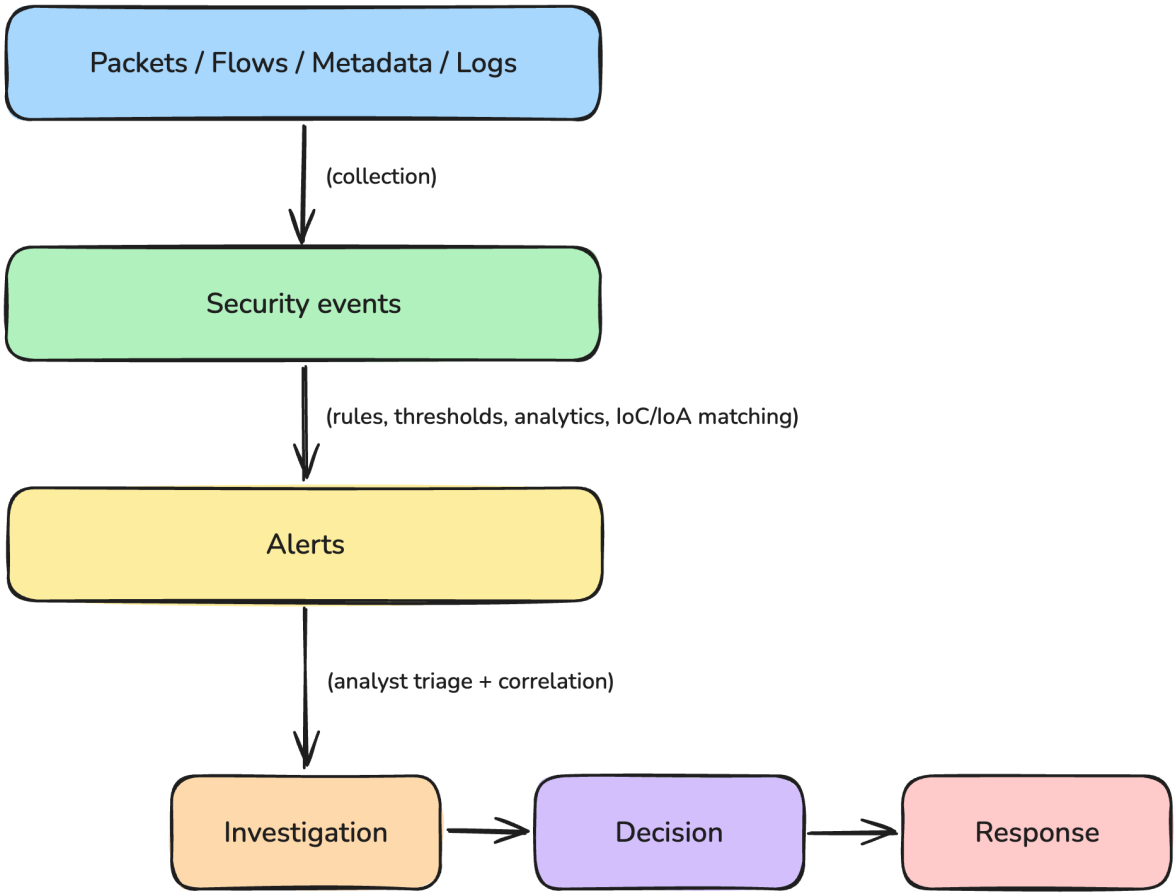


Figure 2.2: How raw network data becomes a security response.

Figure 2.2 shows how raw network data becomes a security response.

3 Network Data Sources

Network Security Monitoring is only as good as the data it collects. This chapter examines three sources: packet captures, DNS traffic and HTTP/HTTPS traffic. For each, it describes what the data looks like, which security events it reveals, its strengths and limits, and the privacy issues it raises.

3.1 Packet Captures (PCAP)

A packet capture records every packet that crosses a monitored interface, including headers and, where present, the payload. It is the most complete form of network evidence, but its value depends on where it is captured, whether packets are dropped, how long it is kept and whether the content is encrypted.

3.1.1 Data Structure

A capture file is a short file header followed by one record per packet. Each record holds a timestamp, the captured and original length, and the raw bytes. The modern pcapng format, Wireshark's default, groups these records into blocks that also describe the capture interface and can store name resolutions and TLS keys. Figure 3.1(a) shows the packet records as an analyst sees them: one row per packet with its timestamp, addresses, protocol and length. Below the list, tools decode each packet into its layers (Ethernet, IP, TCP or UDP, then the application protocol) and show the raw bytes. The full block layout is defined in the IETF pcapng specification [6], and the decoded fields are listed in the Wireshark display filter reference [7].

3.1.2 Identifiable Security Events

- **Evasive scans:** probes with unusual TCP flag combinations, such as NULL or XMAS scans, are visible in individual packet headers.
- **Malware delivery and data theft:** transferred files can be reassembled and hashed, and credentials sent in clear text can be read.
- **Evasion by fragmentation:** split or overlapping packets can be reassembled to expose content hidden from simpler tools.
- **Incident reconstruction:** the exact sequence of requests and responses can be replayed for forensic analysis.

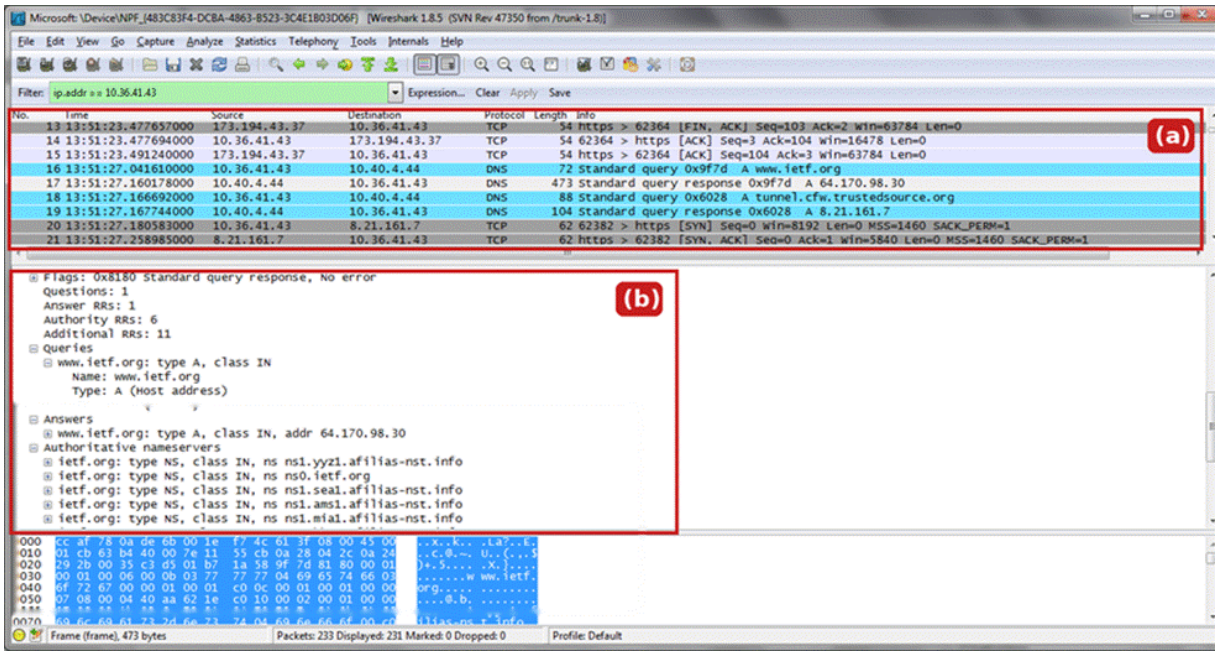


Figure 3.1: One capture viewed in Wireshark: (a) packet records, one row per packet with timestamp, source, destination, protocol and length; (b) the same capture decoded as DNS, showing the query, the answer and the response flags.

3.1.3 Advantages and Limitations

Table 3.1: Advantages and limitations of packet captures.

Advantages	Limitations
The most detailed evidence available; content and files can be reconstructed	Large storage cost, so retention is usually days rather than months
Ground truth for checking what flows, logs and alerts report	High-speed links can drop packets during capture
Independent of any vendor log format	Encrypted payloads cannot be read without the keys

3.1.4 Privacy and Compliance Concerns

Full payloads may contain personal data, credentials and private communications, and a pcapng file that stores TLS keys can expose decrypted traffic to anyone who obtains it. Captures should therefore be kept only as long as needed, truncated or filtered where payload is not required, and protected by strict access control, in line with regulations such as GDPR.

3.2 DNS Traffic

Almost every connection begins with a DNS lookup, so DNS traffic gives a compact record of which names each host tried to reach. It requires a fraction of the storage of full packet capture while still showing behaviour over time.

3.2.1 Data Structure

Each DNS transaction is a query (the name and the record type, such as A or TXT) and a response (a status code, the returned addresses and their time-to-live). A sensor such as Zeek writes one record per transaction with the timestamp, a connection identifier, the client and resolver addresses, the query, the response code and the answers. Figure 3.1(b) shows these fields decoded from the same capture used to illustrate packet records. The complete field list is in the Zeek dns.log documentation [8], and the message format is defined in RFC 1035 [9].

3.2.2 Identifiable Security Events

- **DNS tunnelling and exfiltration:** long, random-looking subdomains or unusual record types such as TXT carry encoded data.
- **Fast-flux infrastructure:** one name resolving to many changing addresses with very low TTLs.
- **Beaconing:** the same name queried at regular intervals by an automated client.
- **Look-alike and algorithm-generated domains:** names imitating trusted brands, or bursts of failed (NXDOMAIN) lookups.

3.2.3 Advantages and Limitations

3.2.4 Privacy and Compliance Concerns

Plain-text queries reveal users' browsing and application use to anyone on the path. Encrypted DNS protects that privacy but removes the organisation's visibility; some browsers, such as Firefox, disable it by default when the local resolver blocks a designated canary domain, which restores monitoring at the cost of user privacy. In addition, the EDNS Client Subnet extension [10] forwards part of the client's address to external name servers, leaking approximate location.

Table 3.2: Advantages and limitations of DNS traffic.

Advantages	Limitations
Compact and structured; easy to search and baseline	DNS over HTTPS or TLS hides queries from network sensors
Links each connection to a domain name	Connections made directly to an IP address bypass DNS entirely
Reveals malicious infrastructure before any payload arrives	Caching means not every connection produces a lookup

3.3 HTTP and HTTPS Traffic

Web traffic shows what users and malware actually do at the application layer: which resources they request, what they download and how they identify themselves.

3.3.1 Data Structure

An HTTP transaction is a request (method, URI and headers such as Host, User-Agent and Referer, optionally with a body) and a response (status code, headers and body). With HTTPS the content is encrypted, so a sensor sees only connection metadata and the TLS handshake: the requested server name (SNI), the TLS version and cipher suites and, in TLS 1.2, the server certificate. Zeek records these in its http, ssl and x509 logs [11]; the protocols are defined in RFC 9110 for HTTP [12] and RFC 8446 for TLS 1.3 [13].

3.3.2 Identifiable Security Events

- **Web application attacks:** SQL injection, cross-site scripting and path traversal in URIs, headers or bodies.
- **Command and control:** requests to a bare IP address, missing or unusual user agents, and regular polling.
- **Malicious downloads and exfiltration:** scripts or executables fetched from unknown hosts, or large uploads to unapproved services.
- **Credential attacks:** repeated 401 or 403 responses, or passwords sent over plain HTTP.

- **Suspicious TLS:** self-signed certificates, a missing SNI or TLS on a non-standard port.

3.3.3 Advantages and Limitations

Table 3.3: Advantages and limitations of HTTP and HTTPS traffic.

Advantages	**Limitations**
Rich application context about user and attacker actions	Most traffic is encrypted; content needs TLS inspection, which is costly and complex
TLS metadata (SNI, certificates, fingerprints) is still visible without decryption	TLS 1.3 encrypts the certificate, and Encrypted Client Hello can hide the SNI
Reveals scanning and exploitation early in an attack	Full content logging needs far more storage than flows or DNS

3.3.4 Privacy and Compliance Concerns

Plain HTTP exposes everything it carries, including personal data and session tokens in URLs and bodies. HTTPS protects the content, but TLS inspection breaks that protection inside the organisation, so decrypted data must be handled under the same rules as any other personal data. Organisations usually exempt sensitive categories such as banking and healthcare from inspection and restrict who can view decrypted traffic.



4 Security Analytics and Detection

4.1 Detection Approaches

Security analytics is the application of detection methods to collected network and log data so that observations become decisions: which host is behaving unusually, how confident the evidence is, and what the analyst should do next. Table 4.1 compares the eight detection approaches used in network security monitoring: what each one matches, what it is good at, and how it fails [17].

The first three are deterministic: they apply prior knowledge — a signature, a policy condition, a number — and return the same verdict every time. The next three learn what normal looks like and flag deviation from it. Correlation and threat intelligence add context and confidence to what the others produce, although a threat-intelligence match can also stand alone as a cheap indicator hit [18].

Table 4.1: Detection approaches compared. The first three depend on prior knowledge, the middle three on a learned baseline, and the last two on context.

Approach	**What it matches**	**Strength**	**Limitation**
Signature	known byte patterns, exploit strings	fast and precise on known threats	blind to new variants and to encrypted payload
Rule	policy conditions (IF-THEN) on fields	deterministic, auditable, aligned with policy	rules grow complex; broad rules flood, narrow ones miss
Threshold	event counts in a time window	simple; catches brute force and scans	low-and-slow stays under; legitimate spikes cross
Statistical	deviation from a historical mean	finds change without a signature	needs history; legitimate change looks anomalous
Behavioural	deviation from a per-entity profile	catches compromised accounts and lateral movement	needs training; roles and schedules change
Anomaly	anything outside a learned baseline	detects novel activity	unusual is not malicious; baseline drift
Correlation	relationships between events across sources	builds a multi-stage narrative; cuts alert noise	depends on consistent fields, clocks and coverage
Threat intelligence	known-bad IPs, domains, hashes, TTPs	fast context on known campaigns	indicators age quickly; infrastructure changes cheaply

No row is sufficient alone. In this project the stager was identified by its payload and hash, but neither says anything about the polling that followed, because the polling carries

no malicious string; a volume threshold caught the HTTP flood but not one request every 5.3 seconds; and the request-level timing test that recovered the poll (cv 0.675) is blind to encrypted, non-HTTP channels (Sections 7.2.9 and 8.3). At flow level the same timing idea flagged a benign TeamViewer relay (cv 0.321) and, once the threshold was loosened, Windows Delivery Optimization (cv 1.203), while missing the implant altogether. Detection quality comes from combining methods, not from tuning one of them.

4.2 Known Indicators vs. Unknown or Previously Unseen Behaviour

Known-indicator detection asks whether an artefact has been seen before. Indicators are concrete — file hashes, domains, IP addresses, URI paths, byte sequences — and matching is cheap, which is why it is the first layer in most SOCs. Its weakness is structural: an indicator describes yesterday's infrastructure. Both C2 certificates in this project were created between four and four and a half hours before first use (15:24:47 and 15:42:40 UTC against first connections at 19:59:46 and 20:00:13), and the stager was addressed by bare IP, so there was no domain to blocklist (Section 8.3).

Behaviour-based detection asks a different question: is this host doing something it does not normally do? It uses volume, duration, port behaviour, timing and peer comparison, and can therefore surface activity that has no indicator at all. The price is a higher false-positive rate, because legitimate change — a new role, a backup window, a software rollout — also deviates from the baseline [15], [16]. Table 4.2 sets the two perspectives against each other.

Table 4.2: Known-indicator and behaviour-based detection compared. They fail in opposite directions, which is why this report uses both.

Aspect	**Known-indicator detection**	**Behaviour-based detection**
Evidence used	hashes, domains, IPs, byte patterns	deviation from a learned baseline
Novel activity	not detected	detected if the deviation is visible
Time to deploy	immediate once the indicator exists	requires a training period
Cost of evasion	low: change address or hash	higher: change the behaviour
Typical error	false negatives on new variants	false positives on legitimate change
Maintenance	feed updates	baseline and threshold tuning

Indicators answer "has this been seen before"; behaviour answers "is this host doing

something new". At 19:45:34 the host contacted a lookalike domain over TLS: a signature had no payload to match, and only a behavioural question had a realistic chance of finding it.

4.3 False Positives and False Negatives

Detection is a classification problem with two error types (Table 4.3). A false positive is benign activity flagged as malicious; a false negative is malicious activity that produces no alert. They are not symmetric in cost: a false positive consumes analyst time, a false negative consumes the organisation's data.

Table 4.3: The two error types in context.

Detection outcome	**Malicious activity**	**Benign activity**
Alert raised	true positive	false positive
No alert	false negative	true negative

False positives come from broad rules, generic signatures, stale baselines, and legitimate activity that resembles an attack. A nightly database backup looks like exfiltration by volume; an authorised vulnerability scan looks like reconnaissance; a software update looks like a download from an unknown host. This project reproduces the pattern: a rule that alerts on every HTTP request to a bare IP address would fire 594 times for one behaviour, and the connection-level timing test flags a benign TeamViewer relay (cv 0.321) and, at a looser threshold, Windows Delivery Optimization (cv 1.203) (Sections 7.2.9, 8.3 and 10).

The operational cost is attention. In an illustrative platform that generates 50,000 alerts a day with 95 per cent false positives [14], an analyst spends the shift discarding alerts, response times rise, and genuine alerts get closed without investigation. That is alert fatigue, and it is a security failure rather than a comfort issue.

False negatives come from the opposite direction: rules too narrow, thresholds too high, no signature for the technique, or deliberate evasion through low-and-slow pacing, obfuscation and encryption. Their cost is dwell time. An attacker who is not detected keeps the access, moves laterally and exfiltrates at leisure. The Equifax case in Section 5 is the extreme version: decryption had been broken for ten months, the traffic was still collected, and nobody acted on it.

Balancing the two is a tuning problem with no perfect solution. Lowering a threshold reduces false negatives and raises false positives; raising it does the reverse. Four controls

move the boundary without pretending to remove it. Context-aware thresholds are tighter on high-value assets and looser on general workstations. Iterative rule tuning removes rules that fire on known-good behaviour and splits rules that conflate several behaviours. Dynamic baselines are refreshed as the environment changes, so a new application does not become an incident. Multi-source correlation requires two independent signals before an alert is raised, which is what turns a weak indicator into a decision.

Both error types remain. The aim is not zero false positives; it is a known, measured rate that the analyst team can carry.

4.4 From Alert to Decision

An analyst does not choose one row of Table 4.1. The methods run in parallel, and their outputs meet at triage. A signature or indicator hit arrives first, but it is a hypothesis: the analyst checks whether the host is expected to talk to that destination, what else it did in the preceding minutes, and whether a second source agrees. Correlation supplies that second source, enrichment supplies the reputation and ownership of the endpoints, and the verdict — benign, suspicious or incident — is recorded together with the evidence that supported it. The order matters less than the rule that no single alert closes an investigation on its own. Section 9 follows one alert through exactly this path.

5 Related Work / Case Study

5.1 Literature Review: Theoretical Foundations of Network Security Monitoring

The theoretical foundation of Network Security Monitoring (NSM) rests on the assumption that perimeter defenses will inevitably be breached, necessitating robust internal visibility. Bejtlich (2013) formally defines NSM not merely as the automated detection of malicious activity, but as the rigorous collection, analysis, and escalation of indications and warnings to detect and respond to intrusions. This is corroborated by the National Institute of Standards and Technology (NIST) Special Publication 800-137, which emphasizes that Information Security Continuous Monitoring (ISCM) is required to maintain situational awareness of all networks and dynamically assess threats (Dempsey et al., 2011).

Recent literature highlights a critical shift from relying solely on static Indicators of Compromise (IoCs)—such as known malicious IP addresses or file hashes—to monitoring for dynamic Indicators of Attack (IoAs) via security analytics (Bhatt et al., 2014). Because attackers can trivially alter their infrastructure to evade signature-based detection, modern NSM relies heavily on statistical and behavioral analysis of network flows, packet metadata, and correlated system logs to identify anomalous communication patterns (e.g., unexpected lateral movement or abnormal data exfiltration volumes).

To contextualize the objectives of this project, the following case studies demonstrate the catastrophic consequences of failures within the NSM lifecycle, specifically concerning network visibility and alert triage.

5.2 Case Study 1: The Equifax Breach (2017) — Visibility Constraints in Encrypted Traffic

The 2017 Equifax data breach, which compromised the personal information of 147 million individuals, serves as a primary case study on the limitations of network visibility when dealing with encrypted application-layer traffic.

The Incident: Adversaries breached the Equifax network by exploiting a known vulnerability (CVE-2017-5638) in the Apache Struts web framework portal. Following the initial compromise, the attackers maintained a persistent presence (dwell time) of 76 days. During this period, they executed over 9,000 queries across 51 different databases and exfiltrated massive amounts of data in small, segmented bursts to evade volume-based

threshold detection (GAO, 2018).

NSM Failure and Analytics Context: The critical failure in Equifax's monitoring architecture pertained to packet capture and HTTP/HTTPS traffic analysis (as outlined in Section 3.1). Equifax possessed an Intrusion Prevention System (IPS) designed to perform deep packet inspection (DPI) on network traffic. However, the SSL/TLS digital certificate required by the appliance to decrypt and inspect encrypted HTTPS network flows had expired 10 months prior to the breach (GAO, 2018).

Consequently, the NSM sensors were "blind" to the application-layer payload. The attackers utilized standard encrypted tunnels to exfiltrate the data, which appeared to the monitoring tools as benign HTTPS metadata. The breach was only discovered when Equifax finally updated the expired SSL certificate, restoring visibility. The security appliances immediately identified the anomalous network flows and generated alerts regarding the suspicious outbound traffic, proving that effective security analytics requires unhindered access to high-fidelity network data.

5.3 Case Study 2: The Target Corporation Breach (2013) — Alert Fatigue and the NSM Lifecycle

While the Equifax breach illustrates a failure in the "Collection" phase of the NSM lifecycle, the 2013 Target Corporation breach highlights a critical breakdown in the "Detection, Decision, and Response" phases, despite successful data collection.

The Incident: Attackers gained initial access to Target's corporate network by utilizing stolen credentials belonging to a third-party HVAC vendor. From there, they moved laterally through the network, bypassing internal segmentation, to access the Point-of-Sale (POS) systems. The attackers deployed custom malware ("BlackPOS") to scrape credit card data from system memory and established an internal staging server to periodically exfiltrate the stolen

6 Laboratory Environment

The practical work used two environments. A generated-traffic lab produced attacks under controlled conditions, so that every packet had a known cause and service impact could be measured. A real-traffic lab processed a public capture of a genuine malware infection, so that detection could be tested on traffic the group did not create (Table 6.1). Full configuration details are given in the appendix.

Table 6.1: The two laboratory environments.

	Generated-traffic lab	**Real-traffic lab**
Purpose	Measure how floods affect a web service	Detect and investigate a real compromise
Data	Traffic generated inside an isolated network	Public 53-minute capture of an infected workstation
Activities	HTTP flood and TCP SYN flood	Malware download, C2 polling, remote access, TLS C2
Ground truth	Known from the experiment schedule	Published exercise answers
Main analysis	Packet analysis, resource and log measurements	Flow analysis, IDS alerts, security dashboard

6.1 Generated-Traffic Lab

Four containers ran on an isolated Podman bridge network (172.28.0.0/24): an nginx and Python web target, an attacker, a benign probe and a sniffer capturing on the target's interface (Figure 6.1). The target was limited to half a CPU core and 64 MiB of memory, and SYN cookies were disabled, so the results apply to that configuration.

6.2 Real-Traffic Lab

The real data is the public 2025-01-22 malware-traffic-analysis exercise, a 53-minute capture of a Windows workstation infected through a fake Google Authenticator download. It was hashed before use, inspected with Wireshark, tshark and Python, and processed by a Docker stack in which Zeek and Suricata generate logs and alerts, Filebeat loads them into Elasticsearch, and Kibana presents the dashboard.

6.3 Safety Controls

No attack traffic or malware could reach a real system. The generated lab had no external route; the malware capture was only read from disk, never replayed, and files

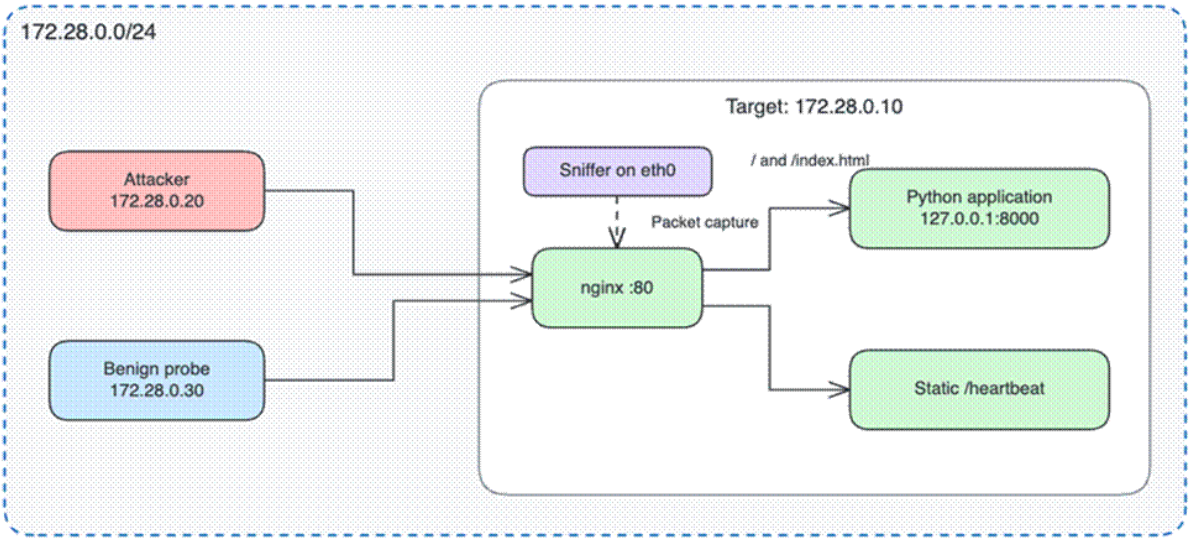


Figure 6.1: Generated-traffic lab topology and capture point.

carved from it were hashed but never opened. Analysis services listened on localhost only, and indicators in this report are defanged.

7 Experimental Methodology

7.1 Packet Analysis

7.1.1 Objectives and Experimental Phases

The investigation compared two mechanisms of service disruption: delayed HTTP processing and failed TCP connection establishment. Packet evidence was correlated with probe outcomes, resource measurements, and nginx logs.

Each run followed four consecutive phases:

Table 7.1: Phases of each packet-analysis run.

Phase	Recorded duration	Activity
Baseline	20 s	Benign probe and local healthcheck
HTTP flood	21.5 s	Probe continues while HTTP flood is generated
SYN flood	32.0 s	Probe continues while SYN flood is generated
Recovery	20 s	Generation stops; monitoring continues

7.1.2 Traffic Generation and Benign Probing

The HTTP generator used hey with 20 concurrent workers, disabled connection reuse, and a configured aggregate target of 1,000 requests/s for 20 seconds. Each request therefore attempted a new TCP connection.

The SYN generator used ZMap against TCP port 80, with a configured rate of 5,000 SYN/s per invocation and source ports drawn from 1024–65535. The orchestration performed 20 invocations with one-second spacing. An attacker-side rule suppressed automatic TCP resets during generation to preserve pending connections. Rule-removal timing was recorded because subsequent resets could affect recovery.

The independent probe requested `/index.html` using exponentially distributed gaps with a mean of one second and a three-second timeout. A probe was successful only when curl completed without error and returned HTTP 200. Connection reuse was checked through TCP stream and handshake inspection.

7.1.3 Packet Collection and Analysis

The sniffer started before baseline and captured traffic using tcpdump.

Capture continued through recovery. The PCAP, capture statistics, phase timestamps, probe results, and server logs were retained for analysis.

Analysis separated traffic by source address, phase, TCP flags, and HTTP fields. Representative connections were inspected to identify handshake completion, request and response timing, SYN/ACK retransmissions, resets, and connection closure.

7.2 Network Flow Analysis

7.2.1 Data and Method

This investigation implements Option B, network flow analysis, on the exercise capture `2025-01-22-traffic-analysis-exercise.pcap` (SHA-256 in Table 7.2; 26,759,675 bytes; 39,427 packets; 19:44:56–20:38:18 UTC). The file was hashed before analysis, no traffic was replayed, and no payload was executed.

Each packet was reduced to one row per 5-tuple: source and destination address, source and destination port, protocol, start time, duration, bytes and packets per direction, and an inferred connection state. The initiator is the sender of the first packet in the conversation — the SYN for TCP, the query for UDP — so `orig_bytes` always means bytes sent by the side that opened the connection. Byte counts are frame lengths and include link and transport headers; they are not payload counts. State is derived from TCP flags, where S0 is an unanswered SYN and SF a completed handshake. The table was built with tshark 4.6.9 and Python 3.11, so every field is explicit rather than trusted to an exporter.

Validation used an independent code path. The flow table reports 421 TCP and 346 UDP conversations, identical to the counts in Wireshark's Statistics → Conversations view for the same file (Figure 7.1); adding 20 ICMP flows gives 787. Zeek reports 859 connections on this capture. The 72-flow gap is a difference in timeout and UDP/ICMP handling that we could not resolve without running Zeek, and it is reported rather than explained away.

Two units are measured. The connection level is what a NetFlow/IPFIX exporter or conn.log would supply. The request level counts HTTP transactions, because one TCP connection can carry hundreds of them; in this capture, 594 requests travel inside three connections. The unit of measurement, not the threshold, decides whether the command-and-control channel is visible at all.

Table 7.2: Data set and tooling. The hash was recorded before analysis, so results are comparable with the other workstreams.

Item	Value
Capture	`2025-01-22-traffic-analysis-exercise.pcap`
SHA-256	`e59db1c07c6fdefafa0abdbca03248c341cdc36c09c34753204d3162802a3586`
Size / packets	26,759,675 bytes / 39,427 packets
Window (UTC)	19:44:56–20:38:18 (53 min 22 s)
Tools	tshark 4.6.9 (Wireshark), Python 3.11, pandas 2.3
Authorisation	Public training capture; no live traffic, no malware executed

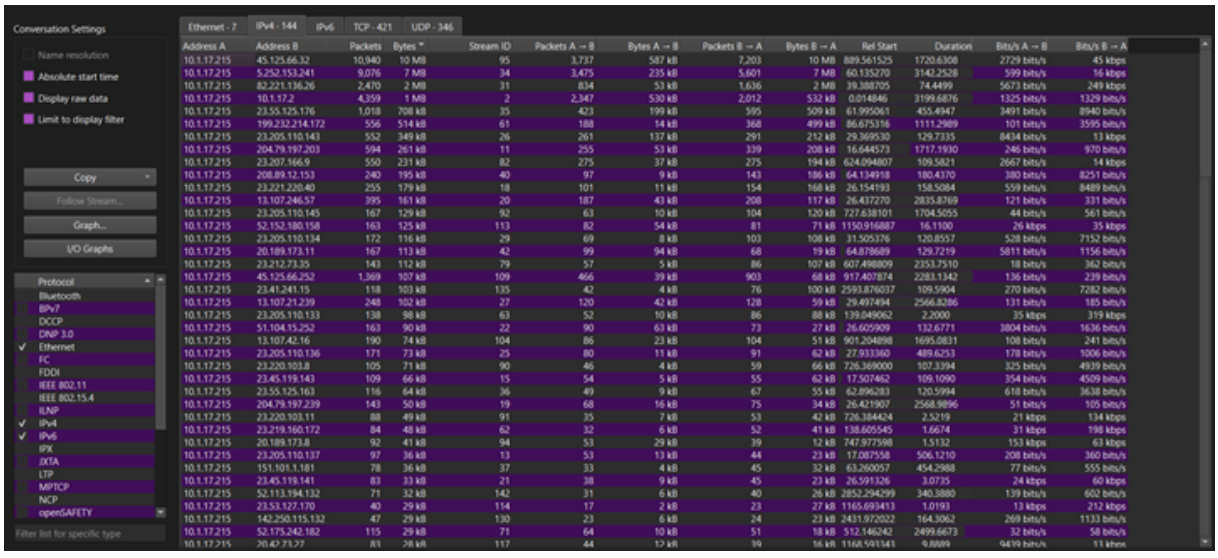


Figure 7.1: IPv4 conversations sorted by bytes, with the TCP 421 and UDP 346 tabs visible. The first two rows are the stager and command-and-control conversations.

7.2.2 Flow Characteristics and Suspicious Pairs

The capture holds 39,087 IPv4 packets; the remaining 340 frames are ARP and other non-IPv4 traffic and were skipped. They reduce to 787 flows — 421 TCP, 346 UDP, 20 ICMP — and one workstation dominates them: 10.1.17.215 accounts for 777 flows to 138 external destinations. A capture-wide average would describe that workstation and nothing else, so every comparison that follows is made per host and per pair.

Port usage separates the host's role from its external activity before any destination is examined (Table 7.3). DNS to the single resolver accounts for 287 flows; Kerberos, LDAP and DCE-RPC to 10.1.17.2 describe a domain-joined Windows workstation. External traffic uses only 80, 443 and 2917. The last two rows carry the intrusion: 19 flows on port 80 held the stager, the downloads and the polling, and nine flows on 2917 carried TLS on a port no browser uses. A monitor that alerts only on 443 sees none of it.

Table 7.3: Destination ports by flow count. A monitor watching only port 443 would miss the stager channel entirely.

Port	Protocol	Flows	Reading
53	UDP	287	DNS to the single resolver 10.1.17.2
443	TCP	239	TLS to 110 destinations: browsing plus C2
389	TCP	80	LDAP to the domain controller
135 / 49668	TCP	28 / 28	DCE-RPC: normal Windows domain traffic
123	UDP	25	NTP
88	TCP	24	Kerberos
80	TCP	19	HTTP: stager, downloads, polling
2917	TCP	9	TLS on a non-standard port

Table 7.4 ranks the five pairs that shaped the investigation. On the 45.125.66.32 pair, 9.69 MB is received against 587 KB sent — roughly 16:1 — so the flow records a download, not exfiltration. The 45.125.66.252 session keeps a connection open for 38 minutes to move 107 KB in total, which is a channel held alive rather than a transfer. The three bursts to 45.125.66.32 begin at 19:59:46, 20:25:16 and 20:28:01, each 7 to 8 seconds after a "PS process started" report to the stager. Flow timing alone cannot prove causation, but the same offset three times is strong circumstantial evidence.

External connections per five-minute bucket fall into four phases: 95 at 19:45 (lure and stager), 100 at 19:55 (TeamViewer), 15 at 20:00 and 58 at 20:25 (command and control). The quiet intervals matter as much as the peaks: the implant is idle until the operator

Table 7.4: Selected pairs. Duration is summed connected time; span is first to last flow, which is why a 34-second total can span 28.7 minutes.

Source → destination	Port	Flows	Bytes out	Bytes in	Duration (span)	Reading
10.1.17.215 → 5.252.153.241	80	3	235,394	6,601,697	3,052 s (52.4 min)	stager; tools delivered, then polled
10.1.17.215 → 45.125.66.32	2917	9	587,077	9,691,463	34 s (28.7 min)	three bursts, ∼3.2 MB received each
10.1.17.215 → 45.125.66.252	443	3	38,961	68,409	2,281 s (38.1 min)	long, near-silent session
10.1.17.215 → 185.188.32.26	80	10	10,304	5,890	6 s	TeamViewer registration
10.1.17.215 → 23.207.166.9	443	25	36,539	194,477	2,715 s	Windows Delivery Optimization (benign)

sends work, then acts within seconds.

Flows give an address and a port, nothing more. Both C2 servers presented self-signed certificates carrying the IP address as the common name, issued at 15:24 and 15:42 UTC on the day of the attack — about four and a half hours before first use. That evidence comes from the TLS layer rather than the flow record and is reconstructed in Section 9.2, Step 4. The largest delivered object was exported and hashed without execution (4,380,968 bytes; SHA-256 in Appendix B), and a second analyst hashing it independently obtained the same value.

7.2.3 Beacon Detection and Evaluation

The conventional flow test scores inter-arrival regularity per source/destination/port triple with the coefficient of variation, CV = standard deviation of the gaps divided by their mean. A value near zero is machine-like. We applied it with the usual minimum of four flows and a threshold of 0.3, and it produced no candidates.

The reason is structural. Of the five destinations the exercise labels attacker-controlled, only one had enough flows to score: 45.125.66.32:2917, nine flows, CV 2.288 — irregular by this measure. The stager pair produced three flows, and the variance of two gaps carries no information. The nearest benign candidate was the TeamViewer registration at CV 0.321: regular enough to rank, and not malicious.

Counted as HTTP transactions, the same stager pair yields 594 events instead of three. The mean gap is 5.30 s over 3,142 s, with CV 0.675. Of the 594 requests, 584 returned 404 and 10 returned 200; 593 carry no User-Agent, and the single exception is the .NET WebClient default. A browser does not ask 584 times for a page that is not there. Polling until work exists is task-queue behaviour, and the rhythm is consistent with an implant rather than a user (Figure 7.2).

Because the exercise publishes its answers, the detector can be scored rather than

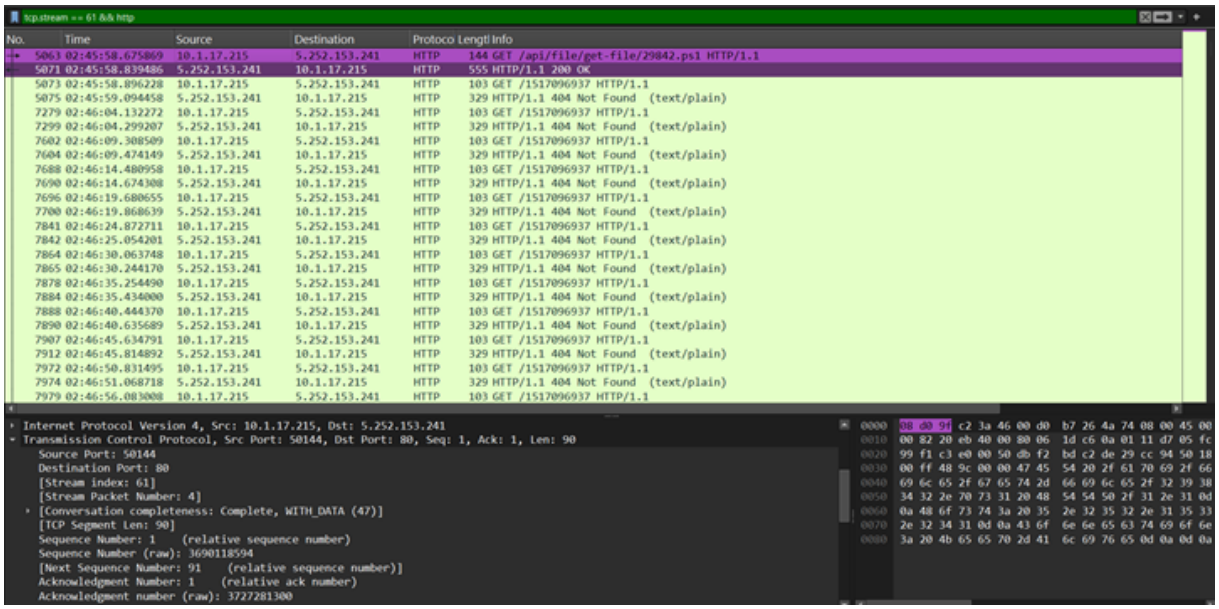


Figure 7.2: One TCP stream (stream 61) carrying hundreds of requests. This is the mechanism that hides polling from flow-level analysis: three flow records, 594 transactions.

described. Five destinations are labelled attacker-controlled; 185.188.32.26 is excluded as a relay, a legitimate TeamViewer service the attacker abused, because counting an ambiguous label would inflate the result. Coverage and recall are reported separately, since collapsing them flatters any detector (Table 7.5, Figure 7.3). At connection level all five pairs are visible in the data, but only one can be scored and none is flagged: detector recall 0.00. At request level only one pair is visible — the HTTP channel — and it is flagged: detector recall 1.00 of one. End-to-end recall, the figure a SOC actually experiences, is 0.00 at connection level and 0.20 at request level.

Table 7.5: Coverage and recall, reported separately. Conflating the two is how a detector can claim high recall while missing the incident.

Quantity	**Meaning**	**Connection level**	**Request level**
Telemetry coverage	malicious pairs visible at this level	5/5	1/5 (HTTP only)
Scoreable coverage	of those, pairs with enough events	1/5	1/1
Detector recall	of visible pairs, flagged	0.00	1.00
End-to-end recall	of all five pairs, flagged	0.00	0.20

The failure is not a tuning problem. From CV 0.10 to 2.00, true positives stay at zero while false positives rise from 0 to 9: lowering the threshold adds noise, not detection. At request level precision is 1.00 at CV 0.75 and above; below that nothing is flagged, because

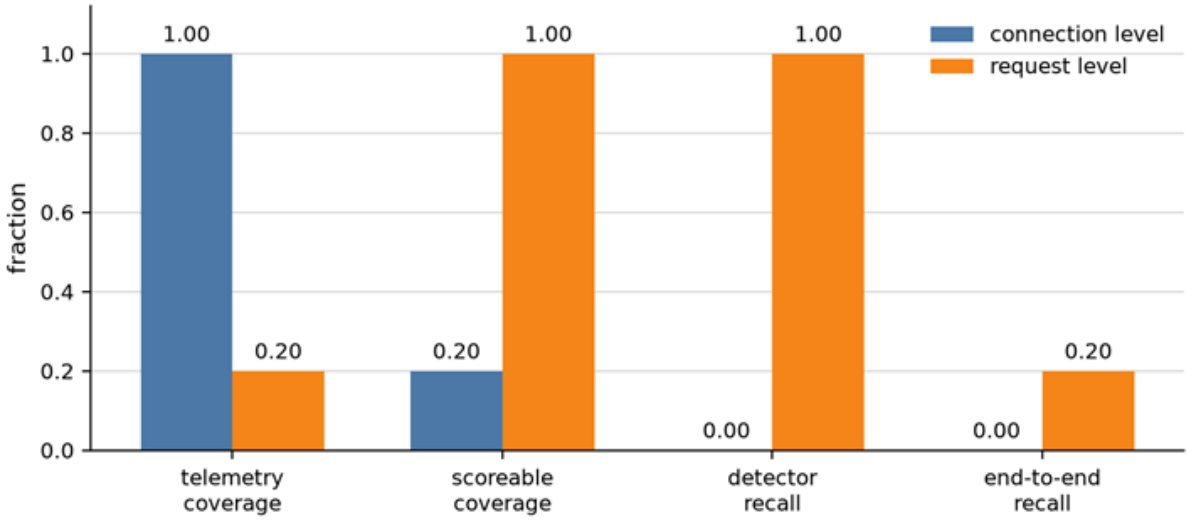


Figure 7.3: Telemetry coverage, scoreable coverage and end-to-end recall at both levels. Conditional columns use a different denominator, because a detector cannot be blamed for traffic its telemetry never sees; the gap between detector recall and end-to-end recall is the cost of missing telemetry.

the polling jitter of 0.675 sits close to the boundary. A tighter-jitter implant would slip through, and we state that as a limit rather than tune around it.

Scored per rule, because an analyst acts on a host and not on a pair, the outcome is more useful than the detector score. The timing rule flagged two pairs and no malicious destination. A rule for ports outside the host's normal set flagged 34 pairs, including 2917, and did recover one. Ranking by inbound volume returned the three largest pairs, all malicious. The host was therefore found by port anomaly and volume, not by timing.

The limits bound the result. Connection-level periodicity testing misses polling inside reused connections, a false-negative class that no threshold removes. 161 triples had fewer than four flows and could not be scored; with two events the standard deviation is undefined, and with three it is unstable. Our table reports 787 flows where Zeek reports 859; the TCP and UDP counts match Wireshark exactly, so the gap lies in timeout or UDP/ICMP handling and remains unresolved. Byte counts are frame lengths, so they exceed payload figures. And the request level sees HTTP only: a C2 carried over TLS with no HTTP layer would be invisible even here.

Flow metadata found the host, the two C2 servers and the three payload pulls without decrypting anything. It did not find the beacon; request-level telemetry did, and that difference is the design constraint the architecture in Section 12 has to meet.

7.3 Security Dashboard Analysis

7.3.1 Objective and Data

This investigation implements Option E, a security dashboard. It processes the same exercise capture used for the flow analysis (`2025-01-22-traffic-analysis-exercise.pcap`) through an SIEM pipeline. The goal is to see whether one dashboard view can point an analyst to the compromised host and the right time window, without first studying packets or flows by hand. The capture is only read from disk. No traffic is replayed onto a network, and files carved from the traffic are hashed but never opened.

7.3.2 Pipeline

The lab runs in Docker Compose on a single host. Zeek and Suricata read the capture offline; Filebeat normalises their output into Elasticsearch, where Kibana presents the dashboard. In parallel, a flow-statistics script scores beaconing and HTTP request rhythm from the Zeek logs. The analyst pivots from any alert back to the original packets by the Zeek connection uid.

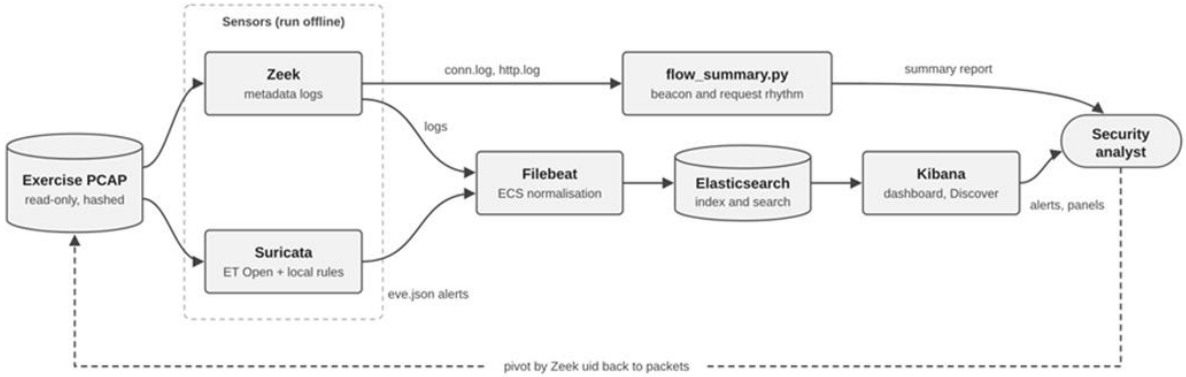


Figure 7.4: Dashboard pipeline. Solid arrows are data flows; the dashed arrow is the analyst pivot. Elasticsearch and Kibana listen on localhost only.

One script (`analyze.sh`) records the PCAP hash, runs Zeek and Suricata offline, and writes the flow summary. Filebeat then picks up the logs automatically. Before using the real capture, the pipeline was checked with a synthetic self-test capture that contains a known beacon and a harmless test request that any working IDS is expected to flag.

7.3.3 Dashboard Design

We built a custom Kibana dashboard, [NSM Lab] Security Analysis, with the panels below. Each panel answers one triage question rather than just counting traffic. Kibana's time zone is set to UTC so that times line up with the PCAP.

Table 7.6: Dashboard panels, chart types and how to interpret them.

Panel	Chart type	How to read it
Traffic summary	Metric	Total connections and bytes for the time window; a quick check that the data loaded completely.
Traffic volume over time	Line chart	Peaks mark bursts of activity; compare them with the alert timeline before investigating.
Top external connections	Data table	Source-destination pairs ranked by bytes, with port, protocol, flows and duration; large volumes or unusual ports stand out.
Top source / top destination addresses	Horizontal bar charts	Show which internal hosts and which external hosts carry most traffic; one dominant internal source narrows the investigation.
Protocol distribution	Pie chart	Shows the protocol mix; a small but unexpected slice (for example HTTP to a bare IP) matters more than the large ones.
IDS alerts by signature	Horizontal bar chart	The most frequent signatures; separates repeated noise from rare, high-value alerts.
Alerts over time	Stacked bar chart	When alerts fire, split by severity; clusters show the phases of an attack.
IDS alerts	Data table	Individual alerts with host, signature and time, used as the starting point for triage.

Table 7.6: Dashboard panels, chart types and how to interpret them (continued)

Panel	Chart type	How to read it
Suspicious HTTP	Data table	Requests to bare IP addresses and script or executable downloads; each row is a lead to check.
TLS with self-signed certificates	Data table	External servers presenting self-signed certificates, with first and last seen times; likely C2 candidates.
HTTP requests per minute	Line chart	A flat, steady line over a long period indicates automated polling; human browsing is irregular.
Data received per minute	Bar chart	Isolated tall bars show large downloads; bars that follow a polling request suggest payload delivery.

7.3.4 Analysis and Rule Tuning

The analyst starts from the alert panels, filters the dashboard to the host that triggered them, and reads the time-based panels for patterns: steady polling, bursts, and long, quiet sessions.

Detection rules were tuned in two passes. The first pass ran untuned local rules, so the number of alerts from each one could be measured. The second pass added a threshold of one alert per host pair per hour to any rule that fired repeatedly on a single behaviour. Every alert on the final dashboard was then labelled true positive, false positive or decoder noise, so the dashboard's precision is measured rather than assumed.

8 Network Traffic Log Analysis

8.1 HTTP Flooding

8.1.1 Observed Pattern

The baseline interval contained 32 probe requests, each using a separate TCP connection. Median SYN-to-SYN/ACK time was 33 μs, while median request-to-first-response-byte time was 11.2 ms, with a 95th percentile of 15.8 ms. These exchanges provided a reference for normal communication within the captured laboratory network.

A subsequent interval contained 1,722 HTTP requests from the attacker to the victim over approximately 20 seconds. The requests repeatedly targeted `/`, with observed rates between 74 and 94 requests/s. Each request used a new TCP connection, producing repeated handshake–request–response exchanges concentrated on the same service.

The packet sequence showed that the victim continued to answer SYN packets promptly. Median SYN-to-SYN/ACK time was 3 μs, indicating that connection establishment was not the main source of delay during this interval.

8.1.2 Response Timing and Service Impact

The delay appeared after the HTTP request. Median request-to-first-response-byte time increased to 217.5 ms, compared with 11.2 ms at baseline. The 95th percentile reached 303 ms, and the maximum reached 400 ms.

Table 8.1: Packet-based timing of the HTTP flood against the baseline.

Packet-based measurement	**Baseline**	**HTTP flood**
Median SYN-to-SYN/ACK time	33 μs	3 μs
Median request-to-first-response-byte time	11.2 ms	217.5 ms
95th-percentile request-to-first-response-byte time	15.8 ms	303 ms

The timing separates fast TCP establishment from slower application responses. It supports a finding of degraded HTTP responsiveness, but the PCAP alone cannot determine whether the delay occurred in nginx, the upstream application, or scheduling and queueing between them.

8.1.3 Connection Closure and Assessment

Among 1,710 connections examined for closure, the victim sent the first FIN on 924 connections, approximately 54%. This distinguishes the interval from baseline, where the

probe initiated closure on all 32 examined connections.

The repeated requests, concentrated connection activity, and increased response delay are consistent with HTTP flooding in the known laboratory scenario. The evidence demonstrates service degradation rather than complete unavailability.

8.2 TCP SYN Flooding

8.2.1 Observed Pattern

The capture contained approximately 100,000 attacker SYN packets sharing a consistent TCP pattern: window size 65,535, a Maximum Segment Size option of 1,460 bytes, and no TCP timestamp option. The packets targeted the victim's port 80 and used 54,195 distinct source ports.

To separate this activity from earlier HTTP connections, the analysis examined 52,485 source ports not previously used during the HTTP interval. Only 1,062 of these ports received at least one SYN/ACK; no SYN/ACK was observed for the remaining 51,423 ports. No completed attacker handshake was identified in the flood traffic.

These counts describe distinct source ports, not individual packet response rates: repeated SYNs using the same port must not be counted as separate connections without further sequence-level analysis.

8.2.2 Incomplete Handshakes

For answered attempts, the victim sent SYN/ACK packets without receiving the final ACK. It subsequently retransmitted SYN/ACK packets, showing that connection establishment remained incomplete.

For example, the connection associated with attacker source port 45733 received SYN/ACK packets at approximately 282.99 s, 284.03 s, 286.08 s, 290.11 s, 298.18 s, 314.56 s.

The successive intervals increased from approximately one second to sixteen seconds. This sequence provides packet-level evidence of repeated attempts to complete the same pending handshake.

8.2.3 Effect on Legitimate Connections

The same interval affected probe traffic from 172.28.0.30. Seven of ten probe connections received no SYN/ACK in the capture. Retransmitted probe SYNs were observed after approximately 1.02 seconds, without a completed handshake for those attempts.

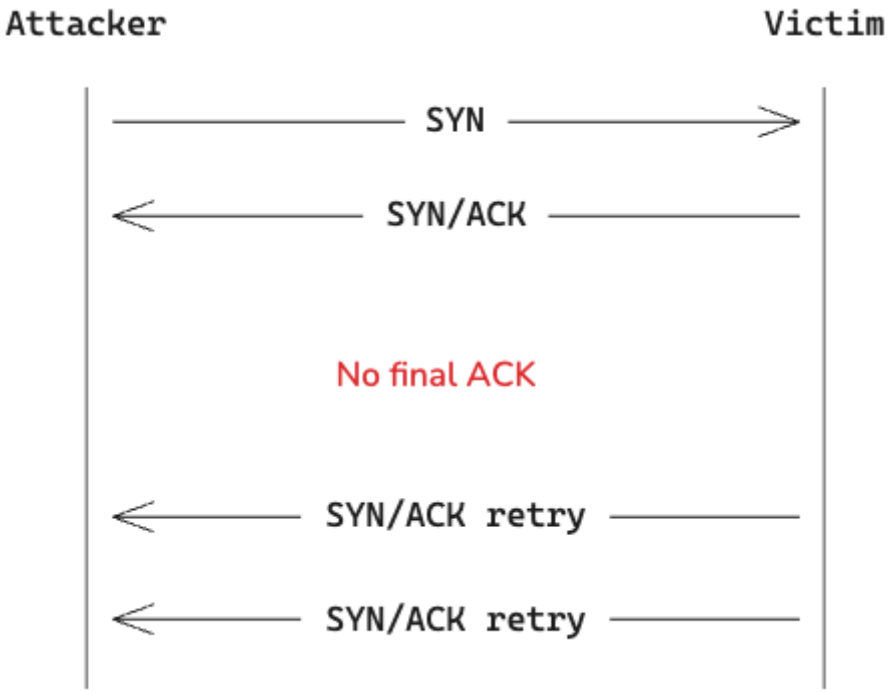


Figure 8.1: Representative incomplete handshake during the SYN-flood interval.

The three successful probe connections had a median request-to-first-response-byte time of 8.8 ms. Thus, the principal observed failure was connection establishment, not delayed HTTP processing after a successful handshake.

8.2.4 Late Resets and Continuing Retransmissions

SYN/ACK retransmissions continued after the final attacker SYN. This exchange shows that some pending attempts were explicitly reset rather than completing normally. Continuing SYN/ACK retransmissions also demonstrate that stopping new SYN traffic did not immediately end all connection-related activity. Exact pending-state counts require socket measurements and cannot be obtained from the PCAP alone.

The combination of concentrated SYN traffic, absent final ACKs, repeated SYN/ACK transmissions, and failed legitimate handshakes is consistent with SYN flooding. Unlike the HTTP activity, where the victim established connections but responded slowly, this activity prevented several sampled clients from reaching the HTTP exchange.

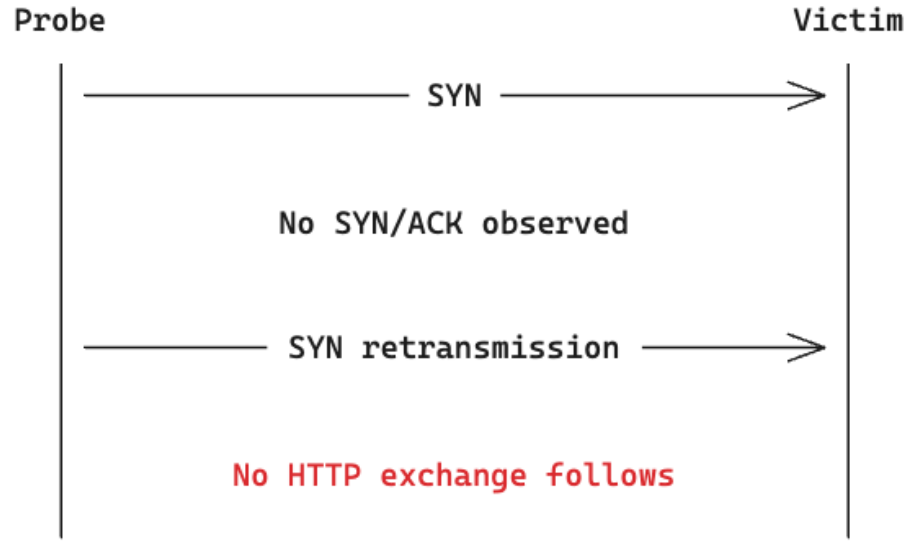


Figure 8.2: Failed probe connection observed during SYN flooding.

8.3 Command and Control Activities

8.3.1 Observed HTTP Activity

Between 19:45:56 and 20:38:18 UTC, the workstation 10.1.17.215 sent 594 HTTP requests to a single bare IP address, 5.252.153.241, on the URI `/1517096937`. No request carried a Referer or a cookie, and 593 of the 594 carried no User-Agent at all. The responses were almost entirely negative: 584 were 404 and 10 were 200. The ten successful responses were the only content the server ever returned. All 594 requests travelled inside three TCP connections, the longest held open for 43 minutes. Section 9 places this channel in the wider intrusion.

Table 8.2: Polling measurements for 10.1.17.215 → 5.252.153.241. The connection count is why a flow log holds three events for this channel (Section 7.2).

Measure	Value
Requests to `/1517096937`	594
Responses	584 × 404, 10 × 200
Mean inter-request gap	5.30 s
Coefficient of variation	0.675
Observation window	19:45:56–20:38:18 UTC (3,142 s)
Requests with no User-Agent	593 of 594
TCP connections used	3 (keep-alive, longest 2,592 s)

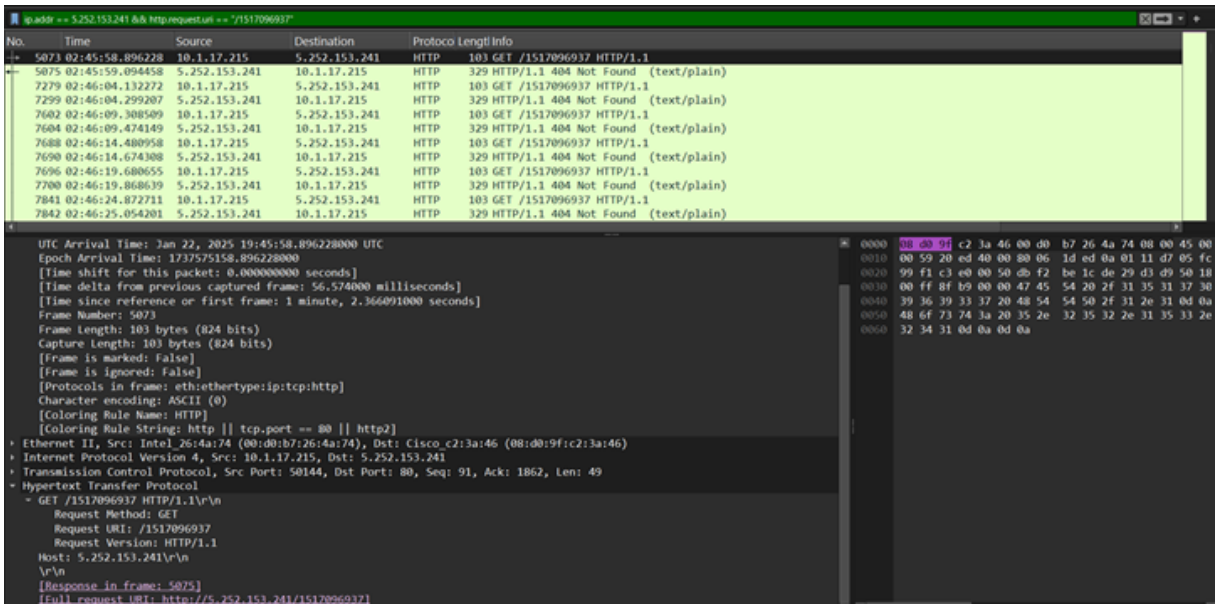


Figure 8.3: Request and 404 response pairs on the polling URI, roughly five seconds apart. The client keeps asking after 584 negative answers, which is task-queue polling rather than browsing.

8.3.2 Analysis of HTTP Behaviour

Three properties make the traffic automated rather than human. The spacing is regular: a mean gap of 5.30 s with a coefficient of variation of 0.675 across 3,142 s, sustained for 52 minutes with no pause for idle time, a meeting or a closed lid. The volume is far above browsing: a user produces tens of requests per page, not 594 to one URI. And the client ignores the answer: a browser that receives 404 stops, while this one continued and only changed behaviour when a 200 arrived. That is task-queue behaviour — ask, be told there is nothing, ask again.

The missing User-Agent points the same way. Legitimate software identifies itself; 593 requests did not. The single exception, the .NET WebClient default string, is itself a fingerprint: it is what a scripting library sends, not what a browser sends.

8.3.3 Detection Significance

The activity is obvious in HTTP metadata and nearly invisible in flow records. Because the 594 requests reused three keep-alive connections, a flow log contains three events, and a connection-level regularity test has nothing to measure; Section 7.2.3 traces that failure across the full threshold range. Detection has to run at request level: count transactions

per host and URI, measure their spacing, and read the response-code mix. The rule needs no signature — more than 100 requests to one destination in an hour, a coefficient of variation below 0.75 and a 404 share above 90 per cent would have surfaced this host.

The indicators have limits. A legitimate long-poll API, an update checker or a misconfigured monitoring agent can produce the same shape, so the pattern ranks a host for review rather than proving compromise. A C2 that uses TLS with no HTTP layer produces no request-level telemetry at all. And part of the evidence here exists only because this malware reported its own progress in clear text inside the URI. The behaviour generalises better than the strings.

8.3.4 Payload and Infrastructure Behind the Channel

The ten successful responses were the only content the channel ever returned, and three of them carried a 354 KB PowerShell agent: frames 19290 and 28332 carry one build, frame 33354 a second. Each is a 36-line script with a Send-Log function, a WebClient-based DownloadString, Convert.FromBase64String and Start-Process, one injected base64 blob and no static URL. Because the endpoints are assembled at runtime, the file itself names no infrastructure; the network record does (Table 8.3).

Table 8.3: Objects returned by the polling channel. Two responses carry the same build; the third differs, so the agent was rebuilt between tasks rather than reused.

Frame	Bytes	SHA-256	Notes
19290	354,353	`1f7d3916...5c0c55bc`	36-line PowerShell agent
28332	354,353	`1f7d3916...5c0c55bc`	identical build, delivered again
33354	354,301	`d63f0163...3704bc`	same pattern, different build

The infrastructure behind the channel was built for this operation. Both servers presented self-signed certificates with the IP address as the common name and placeholder fields (C=XX, L=N/A), issued at 15:24:47 and 15:42:40 UTC on 22 January — four and a half hours before the first connection at 19:59:46. There is no domain to blocklist, no certificate authority to revoke, and nothing in the certificate that predates the campaign; the fields are recorded with the other indicators in Appendix B. An indicator list built the day before would have matched none of it.

That is the practical lesson of this channel: its durable features are behavioural — a bare IP, a numeric URI, 404-dominated polling, agents rebuilt per task — while its addressable features last hours to days. Blocking the address stops this build; detecting the polling pattern stops the next one.

8.4 Indicators of Compromise (IoCs)

8.4.1 HTTP Flood Indicators

The capture identified repeated HTTP requests from attacker 172.28.0.20 to victim 172.28.0.10:80, primarily targeting `/`. Moreover, approximately 1,722 requests appeared within 20 seconds, each using a new TCP connection. Additionally, median request-to-first-response-byte time increased from 11.2 ms at baseline to 217.5 ms, while TCP handshakes remained fast. The combination of concentrated requests and delayed responses provided stronger evidence of service degradation than request volume alone.

For monitoring, these observations support correlating request frequency, repeated endpoint access, connection activity, and response timing. The attacker address and URI can help locate related traffic, but they are not independently malicious IoCs. Authorized load testing or legitimate demand could produce similar patterns and this shows the limitation of the indicators. Therefore, behavioral analysis and contextual evidence are necessary.

8.4.2 SYN Flood Indicators

The capture identified concentrated SYN traffic from attacker 172.28.0.20 to victim port 80, without completed attacker handshakes. After that, the SYN packets shared a TCP window of 65,535, an MSS option of 1,460 bytes, and no timestamp option which made them suspicious. Besides, repeated victim SYN/ACK transmissions showed unresolved connection attempts. These indicators affect seven of ten sampled probe connections, linking the suspicious activity to failures affecting legitimate clients.

For monitoring, these indicators support examining incomplete handshakes, SYN/ACK retransmissions, and legitimate connection failures together. The TCP-option pattern helps group packets within this capture, but it is not a universal attack signature. High SYN counts or missing responses alone are insufficient to establish an attack because retransmissions, network conditions, or capture gaps may produce similar observations. The combined evidence supports SYN-flood-related disruption, not proof of victim compromise.

8.4.3 Malware

An indicator is a fact about an intrusion that can be matched cheaply: an address, a domain, a URI, a hash, a certificate field. This section lists the indicators this capture actually produced, how each one was obtained, and where indicator matching stops being

useful. Nothing below came from a threat-intelligence feed; every value was read from the capture, which is also the test of what a SOC could have extracted on the day.

8.4.4 Indicators Extracted from This Capture

Indicators were derived along four paths, and the path determines how long the result stays useful. Names came from DNS queries and TLS server names; addresses, ports and volumes came from the flow table; URIs and response codes came from HTTP transaction metadata; file hashes were computed in memory from the transferred bytes; and certificate fields came from the TLS handshake. Table 8.4 lists the result with the evidence behind it.

Table 8.4: Indicators from this capture, how each was obtained, and the lifetime a SOC should assume. Values are defanged; full hashes are in Appendix B.

Type	Indicator	Evidence (how obtained)	Operational use and lifetime
Domain	google-authenticator[.]burleson-appliance[.]net; authenticatoor[.]org	DNS query and TLS SNI, 19:45:34 and 19:45:35	DNS sinkhole or proxy deny-list; hours to days
IP	5[.]252[.]153[.]241	594 HTTP requests, 584 × 404 (flow table and HTTP metadata)	Egress block or IDS match; hours
IP:port	45[.]125[.]66[.]32:2917	9 TLS flows, 9.69 MB received, no SNI (flow table and TLS handshake)	Egress deny above 80/443; days
IP	45[.]125[.]66[.]252	3 long-lived TLS flows, one held for 38 minutes	Egress block; hours
URI	`/1517096937`	594 polls, 584 × 404 (HTTP metadata)	Proxy rule and retro-hunt; weeks
URI	`/api/file/get-file/*.ps1`	two script downloads, 1,512 and 1,553 bytes	Proxy or WAF rule; weeks
Hash	`1f7d3916...5c0c55bc`; `d63f0163...3704bc`	SHA-256, computed from the wire, frames 19290, 28332 and 33354	EDR block and retro-hunt; permanent per build
Certificate	CN = IP address, C = XX, L = N/A, self-signed	TLS handshake, notBefore 15:24:47 and 15:42:40 UTC	TLS fingerprint or JA3S; days
Port	2917/TCP	outside the common 80/443 baseline (flow table)	Egress policy; durable locally

Two addresses in this intrusion are not indicators. 104[.]21[.]64[.]1 is a Cloudflare address that fronted the fake download page, and 185[.]188[.]32[.]26 is genuine TeamViewer infrastructure that the attacker merely used. Blocking either would break unrelated services, so both are evidence of the intrusion but not safe rules. Ownership has to be checked before a match becomes a block.

No authentication-failure indicator exists in this capture. Kerberos and LDAP completed normally throughout, and no connection ended in an unanswered SYN; reporting brute-force logons or scanning here would be invention rather than analysis.

8.4.5 Limits of Indicator-Based Detection

Indicators describe infrastructure that existed at the moment of extraction. Both C2 certificates were minted at 15:24:47 and 15:42:40 UTC and first used at 19:59:46 and 20:00:13 — four to four and a half hours of life before contact. Three payloads arrived in two builds, so the hash changed between tasks. A rule written from yesterday's list would have matched none of this, and a rule written from today's covers only this operation.

Content indicators fail under encryption. Both C2 channels used TLS, so no URI, file name or payload was visible; what survived was the certificate, the byte volume, the duration and the timing. A channel that never speaks HTTP leaves no request-level telemetry at all, which is the false-negative class named in Section 8.3.3.

The indicator that actually surfaced the compromised host was not an external one. Port 2917 sits outside the common 80/443 baseline, and top-inbound-volume ranked three malicious pairs, while the timing rule produced no true positive at any threshold (Section 7.2.9). The durable signal was derived from the local environment, not from the attacker's infrastructure. Indicators answer whether this exact artefact has been seen before; at 19:45:34 the question was whether this host was doing something it had never done (Section 4.2). Indicator matching is the cheapest first layer, not a detection strategy on its own.

9 Security Investigation Scenario

9.1 Scenario Setup and Initial Alert

This investigation focuses on the malware-traffic-analysis exercise from January 22, 2025. Monitoring covers a single 10.1.17.0/24 network segment within the bluemoontuesday.com Active Directory domain (BLUEMOONTUESDAY environment). Key infrastructure on this segment includes the default gateway at 10.1.17.1, the broadcast address at 10.1.17.255, and the domain controller at 10.1.17.2 (WIN-GSH54QLW48D).

The case begins with a person, not a sensor: a coworker reports that a colleague downloaded a suspicious file after searching for Google Authenticator, a lure that matches a campaign published by Unit42 on the same day.

The caller provides some information similar to social media posts at:

- https://www.linkedin.com/posts/unit42_2025-01-22-wednesday-a-malicious-ad-led-activity-7288213662329192450-ky3V/
- https://x.com/Unit42_Intel/status/1882448037030584611

At 19:50 UTC on 22 January 2025 the Tier 1 analyst has 26 alerts queued for the last hour, eight of them QUIC decryption noise, when the report arrives. The first machine-generated signal that matches it is a signature hit:

ET DROP Spamhaus DROP Listed Traffic Inbound — 10.1.17.215 to 45.125.66.32:2917. Severity medium, first seen 19:59:46 UTC, repeated three times.

Three properties separate it from the other 25: the source is a workstation rather than a server, the destination port is 2917 rather than 80 or 443, and the destination appears on the Spamhaus DROP list. Each is weak on its own. Together they justify five minutes of triage.

The alert is reconstructed. No IDS ran in this lab, so the signature and the queue are representative rather than logged; every value that follows — addresses, ports, times and volumes — is measured from the capture.

The capture is 26,759,675 bytes and covers 19:44:56–20:38:18 UTC. It holds full packets for the 10.1.17.0/24 segment and the domain controller 10.1.17.2, and the flow and protocol tables were derived from it with tshark 4.6.9. Endpoint and proxy logs are not available (Table 9.1), and that sets the ceiling on what the investigation can prove.

Table 9.1: What the analyst has, and what is missing, at the moment of triage.

Evidence source	**Available?**	**What it provides**
Flow records (Zeek conn.log / flow table)	yes	who talked to whom, volume, duration, port
Packet capture	yes	full payload and timing
HTTP metadata	yes	URIs, status codes, user agents
IDS alerts	yes	signature matches, threat-intelligence hits
DHCP / AD logs (inside capture)	yes	host identity and logged-on user
Endpoint logs (Sysmon, process creation)	no	would confirm execution directly
Proxy logs	no	would confirm browsing to the lure

9.2 Evidence Collection and Investigation

The investigation follows the nine steps the assignment specifies — initial indicator, available evidence, traffic analysis, related logs, suspicious behaviour, alternative explanations, missing evidence, conclusion and response — as four moves: identify the host behind the alert, pivot from the single flow to the host's history, reconstruct the sequence backwards from the first suspicious contact, and test the explanations that would make the traffic benign. Each claim is tied to at least two evidence sources, because one log line is a lead, not a finding.

9.2.1 Host Identification and Traffic Pivoting

DHCP and Kerberos records turn the address into a machine and a person: DESKTOP-L8C5GSJ, MAC 00:d0:b7:26:4a:74, logged on as shutchenson (Steve Hutchenson). Containment targets that laptop and that account, not an address that DHCP may reassign. Only DHCP and Kerberos logs exist inside the capture; firewall, proxy and endpoint logs were not collected, which is why attribution stops at host and account.

The host generates 777 of the capture's 787 flows to 138 external destinations, so the alert is one row in a large history (Section 7.2). Three independent rankings narrow it (Table 9.3): inbound volume puts 45.125.66.32 first with 9.69 MB in nine flows and 5.252.153.241 second with 6.60 MB in three; port rarity puts 2917 alone, outside any workstation baseline. Two unrelated features converge on the same pair within a minute. That convergence, not any single number, is what justifies escalation.

Table 9.2: Host and user attribution from the capture.

Field	Value	Source
IP	10.1.17.215	every suspicious flow
MAC	00:d0:b7:26:4a:74	Ethernet source, DHCP
Hostname	DESKTOP-L8C5GSJ	DHCP hostname and FQDN options
User account	shutchenson	Kerberos CNameString (11 requests)
Full name	Steve Hutchenson	UTF-16 string in directory traffic

Table 9.3: Three independent rankings pointing at the same destination pair.

Ranking	Top result	Reading
Largest inbound volume	45.125.66.32 — 9.69 MB in, 9 flows	matches the alert
Second largest	5.252.153.241 — 6.60 MB in, 3 flows	unexplained bulk HTTP
Non-standard port	2917 only	rare in a workstation baseline

9.2.2 Reconstruction of Suspicious Activities

Working backwards is faster than working forwards: the first contact is easier to recognise than the last. Table 9.4 reconstructs the sequence from the capture, and Figure 9.1 shows the moment the implant reports success at creating persistence.

The chain is complete: lure, download, stager, tooling, remote access, command and control. Every stage is corroborated by at least two sources. The status messages are the implant reporting its own progress in clear text, which is why persistence and execution are described as self-reported rather than proven.

9.2.3 Payload and Communication Analysis

Three of the ten successful responses carried a 354 KB PowerShell agent. Its anatomy is analysed in Section 8.3.4 and its two builds are hashed there; what the investigation adds is timing — each agent arrived within seconds of a "PS process started" report from the implant, so delivery was driven by the attacker, not by the user.

The largest object is the TeamViewer installer, 4,380,968 bytes, beginning `4d 5a 90 00`. A second analyst reproduced its hash independently, which is the strongest check available without executing the file.

Both C2 servers used certificates issued hours before first use — 15:24:47 and 15:42:40

Table 9.4: Reconstructed sequence. Each stage is corroborated by at least two evidence sources.

Time (UTC)	Event	Evidence
19:44:56	capture begins; host already domain-joined	PCAP
19:45:34	DNS and TLS to google-authenticator[.]burleson-appliance[.]net	DNS, TLS SNI
19:45:35	DNS and TLS to authenticatoor[.]org; 2.23 MB in 74 s	conn.log, flow table
19:45:56	first HTTP request to the stager: `/api/file/get-file/264872` (417 B)	frame 5031
19:45:58	`29842.ps1` (1,512 B) fetched; polling of `/1517096937` begins	frames 5063 to 5071
19:47:01	poll returns a task: TeamViewer 4.38 MB, Resource_fr, TV, pas.ps1	frames 8002, 12890, 13643, 13671
19:47:05	client reports “startup shortcut created; status = success”	frame 13677, answered 404
19:55:07	TeamViewer registers with master16[.]teamviewer[.]com	21 requests, DynGate agent
19:59:38	reports “RunRH, status: OK, message: PS process started”	frame 19292
19:59:46	self-signed TLS to 45.125.66.32:2917; 2.6 MB of payload pulled in the first connection	flow and TLS records
20:00:13	self-signed TLS to 45.125.66.252:443, held 1,528 s	flow and TLS records
20:25:16, 20:28:01	two further “PS process started” reports, each followed by a pull within 8 s	frames 28335, 33356

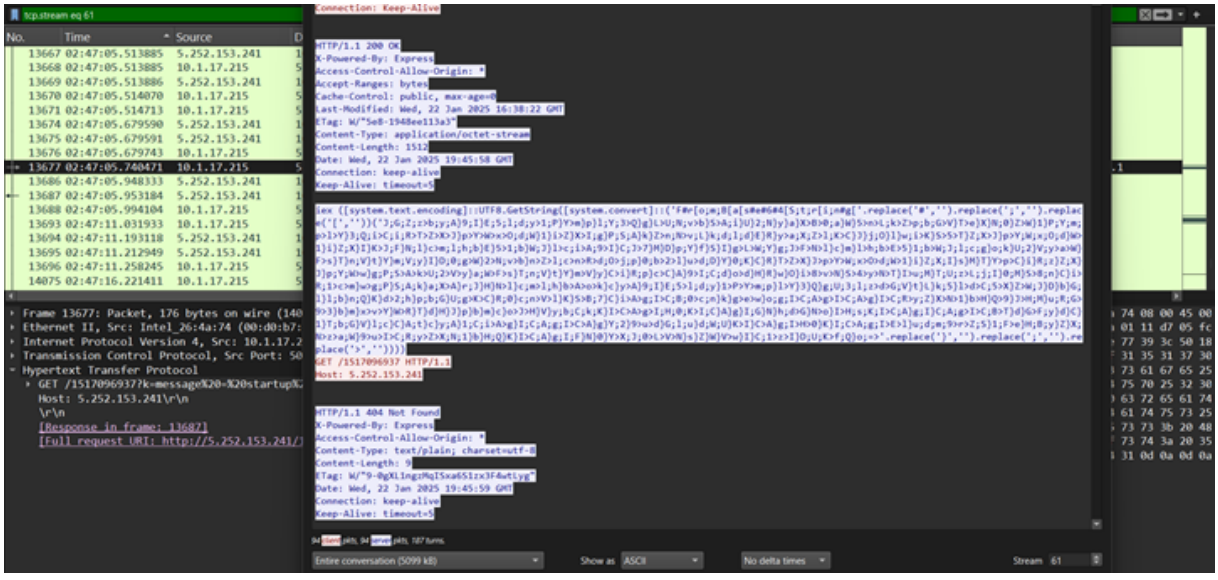


Figure 9.1: The persistence report and its 404 response, with the obfuscated PowerShell loader visible in the same stream view. The implant announces its own success; nothing on the endpoint confirms it.

UTC, against connections at 19:59:46 and 20:00:13. The fields and their lifetime are recorded in Section 8.3.4 and Section 8.4; for the investigation they place the build on the day of the attack rather than in a pool of pre-built servers.

9.2.4 Validation and Differential Diagnosis

A credible investigation explains the benign traffic as well as the malicious. Four candidates were tested (Table 9.5). The most instructive is 185.188.32.26: a genuine TeamViewer registration server contacted by a genuine TeamViewer client that the attacker delivered. Its verdict depends on delivery context, not on the destination; the same address would be unremarkable on a managed desktop where TeamViewer was approved.

Table 9.5: Differential diagnosis. The TeamViewer case shows that verdicts depend on delivery context, not on the destination alone.

Candidate	**Verdict**	**Reasoning**
23.207.166.9:443, 25 flows, 2,714 s	benign	Windows Delivery Optimization; long-lived CDN session, no stager precursor
185.188.32.26, 10 flows	abused, not attacker-owned	genuine TeamViewer registration server; malicious only because the client was delivered by a stager
204.79.197.x, 13.107.x	benign	Microsoft update and telemetry ranges, consistent throughout
8 × QUIC decryption failures	decoder noise	capture lacks TLS keys; not a security event

The same test was applied to the activity itself. A software update would carry an identifying agent and a vendor host; a monitoring agent would follow a fixed schedule rather than 5.3 s with jitter; a user would leave a browser agent and a Referer; a benign long-poll API would return 200 with an empty body rather than 404 for 98 per cent of requests. None of these survives the evidence.

9.2.5 Detection Gap and Remaining Uncertainty

The detector stayed quiet for a structural reason. The 594 polls travelled inside three keep-alive connections, so the flow log held three events and the connection-level test had nothing to score (Section 7.2.9). An analyst who trusted only the detector would have closed this case at the first alert.

Each gap has a specific way to close it, and none of them changes the verdict on the host.

Table 9.6: Residual uncertainty, stated rather than hidden.

Gap	**Why it matters**	**How to close it**
No process-creation logs	execution is inferred from self-reports	Sysmon Event ID 1 on the host
Hashes not checked externally	family attribution	VirusTotal or MalwareBazaar lookup
No proxy or DNS logs	confirm the browsing path to the lure	proxy logs for 19:45
TLS payload not decrypted	the 2.6 MB pulls remain opaque	hash files from the endpoint after containment

9.3 Investigation Conclusion and Response

DESKTOP-L8C5GSJ (10.1.17.215, user shutchenson) was compromised through a fake Google Authenticator download at 19:45:34 UTC. The installer launched a VBScript that started hidden PowerShell, which fetched a stager from 5.252.153.241 and polled it 594 times for tasks. The attacker delivered TeamViewer for remote access and pushed at least three further PowerShell payloads over self-signed TLS to 45.125.66.32:2917 and 45.125.66.252:443. Persistence through a Startup-folder shortcut (TeamViewer.lnk) is self-reported.

Table 9.7: Confidence per claim. Stating confidence is part of the finding, not a hedge.

Claim	**Confidence**	**Basis**
Host compromised and C2 established	high	multiple independent evidence sources
Tooling and payloads delivered	high	sizes, content types and hashes
Persistence created	medium	attacker's own status message only
PowerShell execution succeeded	medium	self-reported; no endpoint telemetry
Data exfiltration occurred	not established	no evidence of outbound bulk transfer

Response follows the four phases of NIST SP 800-61 Rev. 2 (Table 9.8). Containment comes before eradication, and preservation comes before both: the laptop is isolated through EDR but not powered off, the four C2 addresses and the two lure domains are blocked, and egress on 2917 is closed.

Response must not stop at indicators. The infrastructure is cheap to change; the durable lessons are behavioural: HTTP to a raw IP, 404-dominated polling, self-signed TLS on odd ports, and remote-access tooling installed outside software management.

Table 9.8: Response actions by NIST phase.

Phase	**Action**
Containment	isolate the laptop through EDR; do not power it off, to preserve volatile state
Containment	block 5.252.153.241, 45.125.66.32, 45.125.66.252, 82.221.136.26, both fake domains, and egress port 2917
Eradication	reimage rather than clean; remove TeamViewer and the Startup shortcut
Eradication	reset the shutchenson credentials and revoke sessions and tokens
Recovery	restore from a known-good image and re-add to the domain under monitoring
Investigation	hunt the same indicators on all hosts, especially the domain controller 10.1.17.2
Investigation	determine whether the 2.6 MB payloads reached disk; hash from the endpoint
Lessons learned	add alerts for bare-IP HTTP, request-level polling, and self-signed TLS on odd ports from workstations
Evidence	preserve the PCAP and computed hashes with chain of custody

10 Experimental Results

10.1 HTTP and SYN Flooding

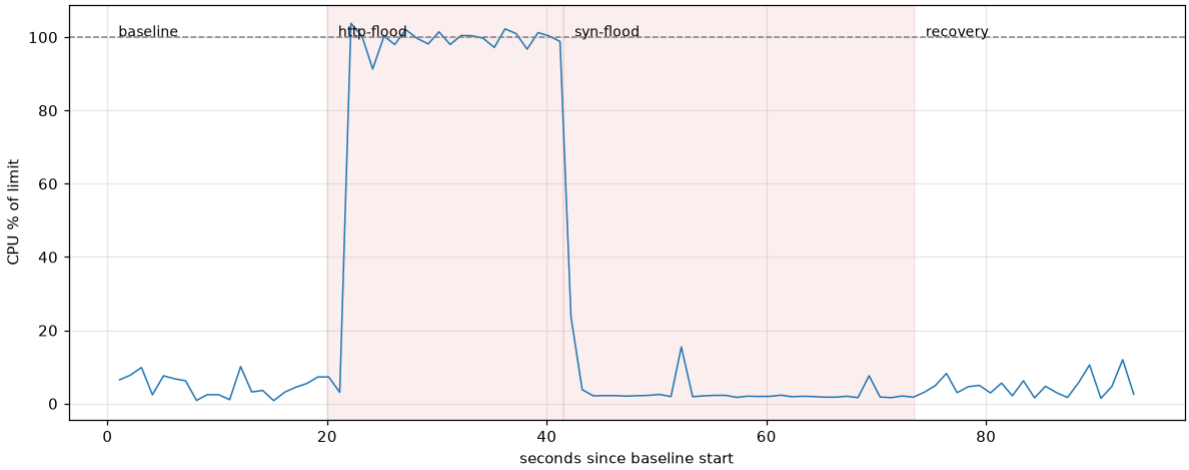


Figure 10.1: Victim CPU utilization across experimental phases.

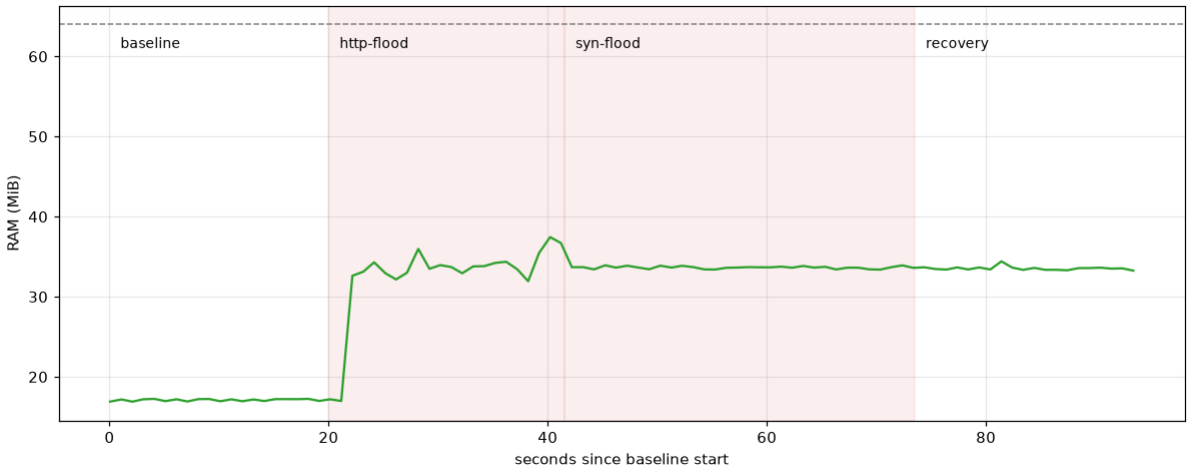


Figure 10.2: Victim memory usage across experimental phases.

10.2 Dashboard Monitoring Results

The dashboard was read over the full capture window (19:44 to 20:39 UTC) in the order an analyst would use it: alerts first, then the host behind them, then the time-based panels. Figure 10.6 shows the complete view, and Table 10.1 summarises what each step revealed.

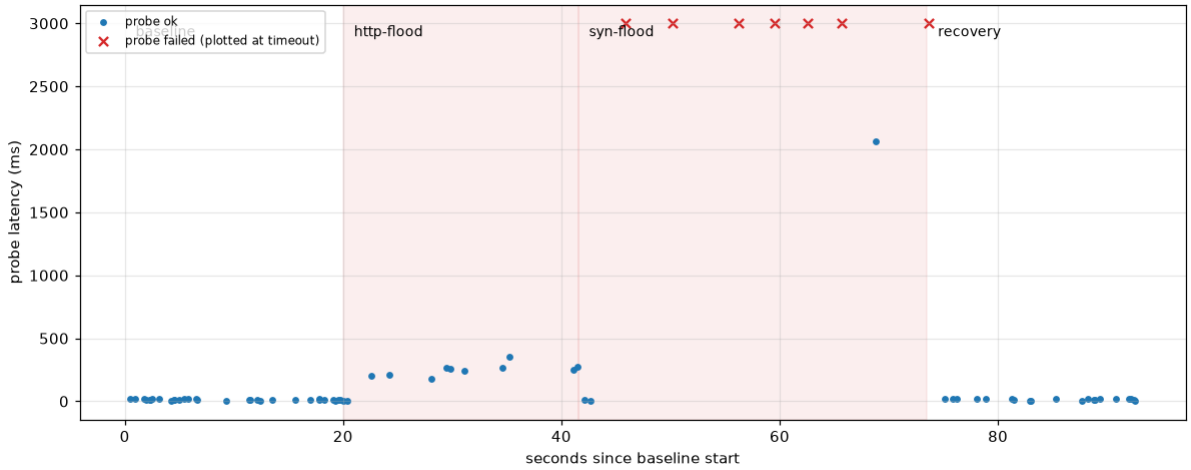


Figure 10.3: Benign probe response times and timeout events.

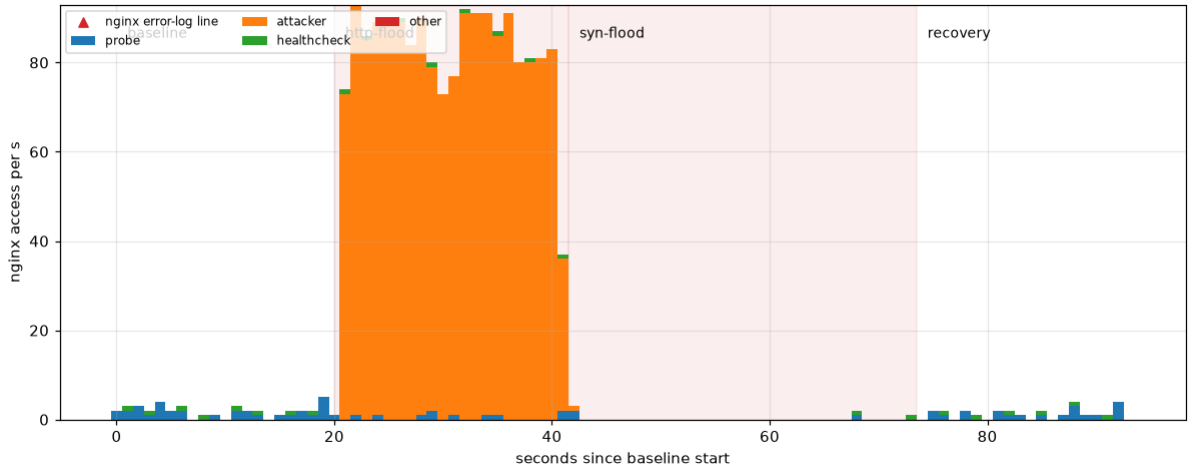


Figure 10.4: Nginx access-log entries per second by client.

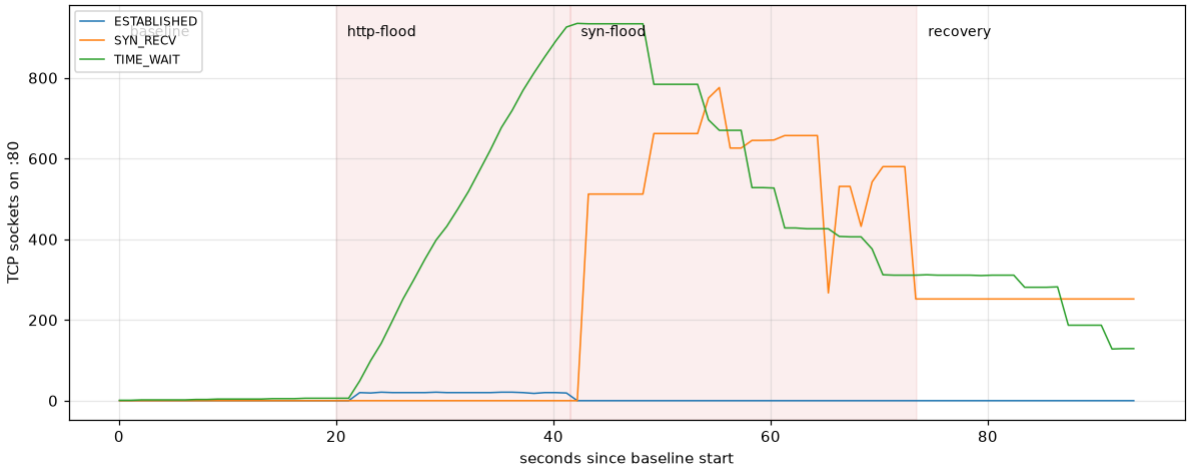


Figure 10.5: Victim TCP connection states on port 80.

Table 10.1: Dashboard reading in triage order.

Triage question	**Panel**	**Observation**
Is there a known threat?	IDS alerts; Alerts by signature	18 alerts from 14 signatures; the PowerShell, Spamhaus DROP and TeamViewer signatures all involve 10.1.17.215.
Which host is affected?	Top source addresses	10.1.17.215 produces 854 flows to 140 destinations; every other internal source is negligible.
Is the activity still running?	HTTP requests per minute	A flat line of about 10 to 11 requests per minute to 5.252.153.241 from 19:46 to 20:37: automated polling, not browsing.
What was delivered?	Data received per minute; Suspicious HTTP	Bursts at 19:45 (installer) and 19:47 (TeamViewer files); .ps1 scripts and executables fetched from a bare IP with no user agent.
Where is the C2?	TLS with self-signed certificates	45.125.66.32:2917 (9 sessions, SNI is the IP itself) and 45.125.66.252:443 (3 sessions, no SNI), both active from 20:00 to 20:28.
What can be closed?	Alerts by signature	Microsoft connection tests, a packed Windows Update file and periodic Microsoft and TeamViewer services.



Figure 10.6: The [NSM Lab] Security Analysis dashboard on the exercise capture, 19:44 to 20:39 UTC.

Clicking 10.1.17.215 in any panel filtered every other panel to that host, so the compromised host, its delivery chain and both C2 servers were identified from one screen before any packet was opened. The packets were then needed only to confirm details such as the URI status messages and file hashes.

10.3 From Indicators to Detection Rules

Each indicator extracted from the capture was turned into a rule at the layer that can actually see it. Atomic indicators (addresses, domains, hashes) become exact-match rules that are precise but short-lived; behavioural indicators become rules on patterns that the attacker cannot change as cheaply (Table 10.2).

Table 10.2: Indicators translated into rules, ordered from atomic to behavioural.

Indicator	Visible in	Rule	Expected lifetime
google-authenticator[.]burleson-appliance[.]net, authenticatoor[.]org	DNS query, TLS SNI	Suricata match on dns.query and tls.sni	Days
5[.]252[.]153[.]241	Every packet	IP-list rule; egress block	Hours to days
45[.]125[.]66[.]32, 45[.]125[.]66[.]252	Every packet	IP-list rule (already covered by Spamhaus DROP)	Days
`/1517096937`, `/api/file/get-file/`	HTTP request	Suricata content match on http.uri	Weeks
SHA-256 of delivered files	Zeek files.log	Kibana indicator-match rule on the file hash	One build
HTTP to a bare IP with no user agent	HTTP request	Suricata http.host pattern, thresholded per host pair	Durable
Self-signed TLS on a non-standard port	Zeek ssl.log, conn.log	Kibana query rule on certificate status and port	Durable
Regular polling of one host	Zeek http.log	Request-rhythm analytic: at least 20 requests to one host with a coefficient of variation below 1.0	Durable

Typical rules for the atomic and behavioural cases are shown below (simplified; local rules use SIDs from 9000001):

```
# Atomic: stager polling URI
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"LAB IoC stager polling URI";
 flow:established,to_server; http.uri; content:"/1517096937"; startswith;
 threshold:type limit, track by_both, count 1, seconds 3600; sid:9000010; rev:1;)

# Atomic: look-alike download domain
alert dns $HOME_NET any -> any any (msg:"LAB IoC look-alike authenticator domain";
 dns.query; content:"authenticatoor.org"; nocase; endswith; sid:9000011; rev:1;)
```

```
# Behavioural: HTTP request to a bare IP address, one alert per host pair per hour
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"LAB HTTP request to raw IP host
    ";
 flow:established,to_server; http.host; pcre:"/^\d{1,3}(\.\d{1,3}){3}(:\d+)?$/";
 threshold:type limit, track by_both, count 1, seconds 3600; sid:9000001; rev:2;)

# Behavioural (Kibana KQL): self-signed TLS from the LAN on a non-standard port
event.dataset:"zeek.ssl" and zeek.ssl.validation.status:"self signed certificate"
 and not destination.port:(443 or 8443)
```

The threshold matters as much as the match. The first version of the raw-IP rule had no threshold and alerted on every one of the 594 polling requests; limiting it to one alert per host pair per hour kept the detection and removed 593 redundant alerts.

10.4 Rule Set Evaluation

Every alert on the final dashboard was labelled true positive, false positive or ambiguous against the exercise answers, and coverage was measured per attack stage rather than per packet. One caveat applies: rules built from this capture's own indicators are bound to fire on it, so their score here measures correctness, not generalisation. Only the behavioural rules and the ET Open signatures were written without knowledge of this attack.

10.4.1 Alert Volume

Table 10.3: Alert volume before and after tuning.

Stage of tuning	Alerts	Change
ET Open and untuned local rules	619	594 from the raw-IP rule alone
Raw-IP rule thresholded per host pair per hour	26	−593
QUIC decryption failures excluded as decoder noise	18	−8

10.4.2 Alert Verdicts

Precision is therefore 12 of 18 (0.67), or 12 of 17 (0.71) if the ambiguous alert is set aside. Every false positive came from a low-severity informational signature or from the connection-level beacon rule; none came from a rule built on an indicator of this attack.

Table 10.4: Verdicts for the 18 dashboard alerts: 12 true positives, 5 false positives, 1 ambiguous.

Signature group	Alerts	Verdict
Script and executable delivery (ET ATTACK_RESPONSE, ET INFO PS1, ET HUNTING, ET INFO Executable Retrieved, LAB PowerShell script)	8	True positive: stager scripts and TeamViewer files from 5.252.153.241
ET DROP Spamhaus DROP Listed Traffic	2	True positive: both C2 servers
LAB HTTP request to raw IP host	1	True positive: the polling channel
ET INFO TeamViewer Dyngate User-Agent	1	True positive in context: legitimate tool installed by the stager
ET INFO PE EXE or DLL Windows file download	1	Ambiguous: matches both the TeamViewer delivery and Windows Update pattern; needs the file hash to decide
ET INFO Microsoft Connection Test	2	False positive: Windows network-status check
ET INFO Packed Executable Download	1	False positive: Windows Update file
LAB Possible beaconing	2	False positive: Windows Delivery Optimization and the TeamViewer relay

10.4.3 Coverage by Attack Stage

Table 10.5: Detection coverage per attack stage: 5 of 7 stages raised an alert.

Attack stage	Alerted?	Rule that fired	Outcome
Lure: look-alike domains	No	None (new domains, no reputation)	False negative
Stager download	Yes	PowerShell signatures	True positive
Tool delivery	Yes	Executable signatures	True positive
Persistence report	No	None (clear-text URI only)	False negative
Remote access	Yes	TeamViewer Dyngate	True positive (context)
HTTP polling	Yes	Raw-IP rule; request-rhythm analytic. Connection-level beacon rule missed it	True positive; false negative for the beacon rule
TLS C2	Yes	Spamhaus DROP	True positive (reputation only)

10.4.4 False Negatives and False Positives

The false negatives share a cause: the first contact and the encrypted channel carry no content a signature can match.

- **Lure domains** were registered for the campaign, so no reputation list or signature knew them. Only the indicator rule, written afterwards, would catch a repeat.
- **Polling inside keep-alive connections** produced three flow records for 594 requests, so the connection-level beacon rule had nothing to score. The request-rhythm analytic on http.log found it.

- **TLS C2** was detected only because Spamhaus had already listed both servers. Their certificates were issued hours before use, so a fresh server would pass unflagged; the self-signed-certificate rule is the durable replacement.
- **Persistence** is visible only because the malware reported it in clear text. Without endpoint logs, a quieter implant would leave no network trace of this stage.

The false positives are cheaper to fix (Table 10.6).

Table 10.6: False positives and the tuning applied to each.

False positive	**Why it is benign**	**Tuning**
Microsoft Connection Test	Windows checks internet access to a Microsoft host	Keep at low severity and off the alert queue for managed hosts
Packed Executable Download	Signed Windows Update file from a Microsoft CDN	Allow-list Microsoft update domains; alert only on unsigned files
LAB Possible beaconing	Periodic Microsoft and TeamViewer services	Allow-list known service ranges and require a second signal, such as a bare-IP host or a rare destination

Overall, signatures and reputation lists covered the tooling stages well, behavioural rules were needed for the polling channel, and the remaining gaps sit where the attack was new or encrypted. Tuning cut the alert volume from 619 to 18 without losing a single true detection, which is the result that matters for an analyst's queue.

11 Security Analysis and Discussion

This chapter brings the findings from both data sets together to answer three questions: how suspicious behaviour can be told apart from normal behaviour, how separate pieces of evidence can be correlated into a finding, and how well security analytics identified the suspicious behaviour. Three activities are used throughout: the HTTP flood and the SYN flood from the controlled lab, and the command-and-control (C2) polling from the public malware capture.

11.1 Distinguishing Normal from Suspicious Behaviour

No single value in this project was suspicious on its own. Suspicion came from deviation from a baseline that fits the host's role, and from combinations of features that legitimate software rarely produces together (Table 11.1).

Table 11.1: Normal and suspicious behaviour for the three activities.

Activity	**Normal reference**	**Observed**	**Benign look-alike**
HTTP flood	Probe requests about once a second; median time to first response byte 11.2 ms	1,722 requests in about 20 s from one source to one URI; median time to first byte 217.5 ms while handshakes stayed fast	Authorised load test or a genuine surge in demand
SYN flood	Handshakes complete in microseconds and probe connections succeed	About 100,000 SYNs from 54,195 source ports, no final ACK, SYN/ACKs retransmitted at 1, 2, 4, 8 and 16 s; 7 of 10 probe connections failed	Packet loss, a congested link or gaps in the capture
C2 polling	A workstation browses irregularly, with a user agent and a Referer	594 requests to a bare IP about 5.3 s apart (CV 0.675), 584 answered 404, almost all without a user agent	Update checkers, long-poll APIs and monitoring agents

Two lessons follow. First, the baseline has to be specific to the host's role. A workstation polling a bare IP address or opening TLS on port 2917 is unusual; the same rhythm from a monitoring server would not be. Second, the benign look-alikes are real rather than theoretical. The connection-level timing test flagged Windows Delivery Optimization and a TeamViewer relay, both legitimate, and an authorised load test would look exactly like the HTTP flood in the packets. What separated the attacks from these cases was a combination of features: regularity together with a bare-IP host, 404 answers and no browser identity for the polling; volume together with degraded service for legitimate users for the floods.

The two floods also show that "suspicious" must be defined by impact as well as by

traffic shape. In the HTTP flood the server kept accepting connections quickly but answered slowly, so the damage sat in the application. In the SYN flood the HTTP exchange was fast for the few clients that got through, but most never completed a handshake. The same label, denial of service, covers two different failures that need different detection and different mitigation.

11.2 Correlating Multiple Pieces of Evidence

Each data source saw only part of each activity. A finding was accepted only when at least two independent sources agreed, and the correlation usually added something that no single source could show (Table 11.2).

Table 11.2: Findings confirmed by correlating independent sources.

Finding	**Evidence A**	**Evidence B**	**What correlation adds**
HTTP flood degraded the service	PCAP: time to first byte rose about twentyfold while SYN/ACK stayed fast	Probe response times, nginx log rate and victim CPU over the same phase	Places the delay in the application rather than the network
SYN flood blocked new connections	PCAP: SYNs without a final ACK and repeated SYN/ACKs	Failed probe connections and TCP connection states on port 80	Shows that connection establishment, not HTTP processing, failed
10.1.17.215 is compromised	Suricata: PowerShell stager and Spamhaus DROP alerts	Zeek http.log: 594 polls to a bare IP; DHCP and Kerberos: host DESKTOP-L8C5GSJ, user shutchenson	Turns an IP address into a machine and an account to contain
The stager drives the TLS C2	http.log: three “PS process started” reports	conn.log and ssl.log: a self-signed TLS transfer from 45.125.66.32 within 7 to 8 s of each report	A timing link between two channels that no single log contains
TeamViewer is malicious here	files.log: TeamViewer executables served by the stager	http.log: DynGate registration with genuine TeamViewer servers	Context turns a legitimate tool into evidence

Correlation depends on join keys more than on tools. In this project the keys were a common UTC clock, the IP-to-host mapping from DHCP, the Zeek connection uid shared across conn, http, ssl and files logs, and the flow identity that links an alert back to its packets. Where a key was missing, correlation stopped. With no endpoint logs, the persistence shortcut and the PowerShell execution could only be inferred from the malware’s own clear-text reports, so they remain medium-confidence findings.

Correlation also removed false positives. Windows Delivery Optimization was regular enough to trip the timing rule, but it had no stager before it, used a Microsoft CDN host and carried a normal client identity. The TeamViewer relay was genuine infrastructure, and only the fact that the client had been delivered by the stager made it relevant. In both cases a second source changed the verdict.

11.3 How Security Analytics Identified Suspicious Behaviour

Each analytic method found a different part of the activity and missed another (Table 11.3).

Table 11.3: What each analytic method found and missed.

Method	What it found	What it missed
Signatures	Stager scripts, executable delivery, TeamViewer registration	Lure domains, persistence, anything inside TLS
Reputation lists	Both TLS C2 servers (Spamhaus DROP)	Infrastructure not yet listed; C2 certificates were only hours old
Volume and thresholds	Both floods; the largest inbound pairs	Low-and-slow polling at about 11 requests per minute
Connection-level timing	No malicious pair; two benign services flagged	Polling hidden inside three keep-alive connections
Request-level timing	The polling channel (594 requests, CV 0.675)	A C2 channel with no HTTP layer
Port anomaly	TLS on port 2917	The second C2 server, which used port 443
Dashboard correlation	The host, the delivery chain and both C2 servers on one screen	Only what the sensors already produce

Three conclusions stand out. The unit of measurement mattered more than the threshold: no setting of the connection-level timing test found the polling, while counting requests found it immediately. Thresholds without tuning overwhelm the analyst: one untuned rule produced 594 alerts for one behaviour, and tuning reduced the queue from 619 to 18 alerts without losing a true detection. Finally, the methods fail in different places, so they are useful together and weak alone. The tuned rule set reached a precision of about 0.7 and raised an alert in 5 of the 7 attack stages; the stages it missed were the new lure domains and the self-reported persistence, which no network signature could have known in advance.

11.4 Limitations

- **Clean ground truth.** The malware capture contains one infected host and a published answer key, which makes labelling easier than in a real network.
- **Configuration-specific lab results.** The flood results depend on a target limited to half a CPU core with SYN cookies disabled; a production server would degrade differently.
- **Optimistic indicator rules.** Rules built from this capture's own indicators are bound to match it; only the behavioural rules and public signatures were tested on unseen behaviour.

- **Missing sources.** No endpoint, proxy or firewall logs were available, and TLS was not decrypted, so execution and payload content remain inferred.

Within these limits, the project answers its central question. Network traffic and security data can identify suspicious activity reliably when behaviour is compared with a role-specific baseline, when every finding is confirmed by a second independent source, and when signature, reputation and behavioural analytics are layered so that each covers the others' blind spots.



12 Proposed Monitoring Architecture

This section proposes a network security monitoring (NSM) architecture for a small hypothetical organisation and explains how information flows through it.

12.1 Scope and Goal

12.1.1 Hypothetical Organisation

XYZ Ltd is a small office in Ho Chi Minh City, Vietnam with about 40 staff. All staff use Windows workstations on a single office LAN. The LAN reaches the internet through one firewall, which also performs NAT. There is no in-house server room: email and documents are provided by cloud services. One IT administrator is responsible for security and acts as the security analyst.

12.1.2 Goal

The goal of the architecture is to protect the company's staff, data and business operations by giving continuous visibility of the network, so that attacks are detected early, investigated with reliable evidence and stopped before they cause serious damage.

The architecture is designed to detect the threats most likely to affect a small office:

Table 12.1: Threats the architecture is designed to detect, and how each appears on the network.

Threat	**How it shows up on the network**
Phishing and malicious downloads	Visits to newly registered or look-alike domains, executable or script files downloaded over the web
Malware and remote-access tools	Regular connections from a workstation to the same external server (command-and-control beaconing)
Ransomware	Contact with known-bad infrastructure before encryption starts, and unusual traffic volumes
Data theft	Large or long-lasting uploads to unfamiliar external destinations, often outside working hours
Network scanning and exploitation attempts	Many connections to different ports or hosts in a short time, IDS signature matches
Policy violations	Unapproved services such as remote-access software, peer-to-peer or anonymising tools

To meet this goal, the architecture has four objectives:

1. **Visibility:** record every connection between the office network and the Internet, with enough detail to know which host talked to which destination, when, how often, for how long, and how much data was exchanged.

2. **Detection:** raise timely, prioritised alerts for both known threats (signatures, IoCs) and unknown ones (unusual flows, beaconing, rare domains).

3. **Investigation:** let the analyst move from an alert to the related flows, DNS queries and original packets, so that every decision is based on evidence.

4. **Response and improvement:** support containment and recovery, and feed what is learned from each incident back into detection.

12.1.3 Scope

Network segments: the office LAN and its traffic to and from the Internet are in scope. The server VLAN, DMZ and guest Wi-Fi are out of scope.

In this design, we only focus on the data sources that we mention in previous sections, and the others are out of scope.

Table 12.2: Data sources in and out of scope.

Data source	In scope	Collected by
Packet captures	Yes	tcpdump on the network sensor
DNS traffic	Yes	Zeek dns.log
HTTP/HTTPS traffic	Yes	Zeek http.log and ssl.log (HTTPS as metadata only, no decryption)
TCP/UDP flows	Yes	Zeek conn.log
IDS/IPS alerts	Yes	Suricata in passive IDS mode; inline IPS blocking is out of scope
Firewall logs	Yes	Firewall syslog export
Authentication logs	No	Out of scope
Network device logs	No	Out of scope (switch and router logs)
Endpoint logs	No	Out of scope (no EDR or host agents)

12.2 Architecture Diagram

The architecture has seven layers, read from top to bottom:

1. **Traffic sources:** the office LAN, the firewall and the internet. A test access point (TAP) sits on the link between the LAN and the firewall.

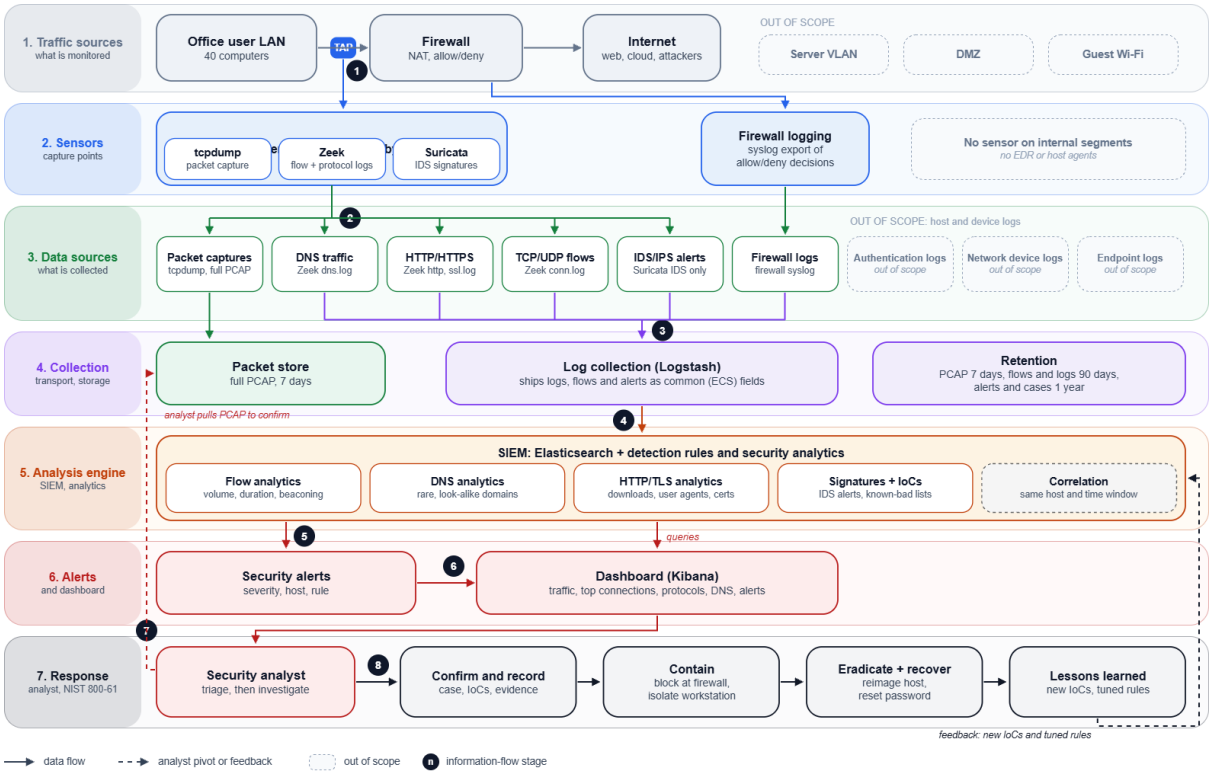


Figure 12.1: Proposed network security monitoring architecture for a small office. Grey dashed items are out of scope; solid arrows are data flows; dashed arrows are analyst pivots and feedback; numbered circles are the stages of the information flow in Section 12.4.

2. **Network sensors:** a passive network sensor fed by the TAP (tcpdump, Zeek, Suricata), and the firewall's own logging.

3. **Data sources:** the six in-scope data sources produced by the sensors. The three out-of-scope sources are shown greyed out.

4. **Collection:** the packet store keeps full PCAP; Filebeat ships all logs, flows and alerts to the SIEM.

5. **Analysis engine:** the SIEM runs flow, DNS, HTTP/TLS, signature and correlation analytics.

6. **Alerts and dashboard:** rule matches become security alerts, shown with traffic context on the dashboard.

7. **Response:** the security analyst triages and investigates, then follows the incident-response steps. Lessons learned feed back into the SIEM.

12.3 Analysis: Flows, Packets and Signatures Together

The SIEM combines three kinds of analysis on the same data, matching the detection approaches in Section 4:

Table 12.3: Analysis performed by the SIEM.

Analysis	Data sources	What it detects
Flow analytics	Zeek conn.log, firewall logs	Top talkers, unusual volume, long connections, regular beaconing
Signatures and IoCs	Suricata alerts, threat-intelligence lists	Known malware traffic, known-bad IP addresses and domains
DNS analytics	Zeek dns.log	Rare or look-alike domains, connections to IPs never resolved by DNS

Flow analysis is cheap and works even when traffic is encrypted, so it runs on everything and is kept for 90 days. Packet capture is expensive, so it is kept for 7 days and used mainly to confirm what the flows and alerts point to.

12.4 Information Flow and Process

Information moves through the architecture in eight stages.



1. **Capture.** Workstation traffic passes through the TAP on its way to the firewall. The TAP sends an exact copy of every packet to the sensor, and the firewall logs onto each connection it allows or blocks.

2. **Sensing.** On the sensor, tcpdump stores the full packets in the packet store, Zeek turns them into flow records (conn.log) and protocol logs (dns.log, http.log, ssl.log), and Suricata matches them against signatures.

3. **Collection.** Logstash collects the Zeek logs, Suricata alerts and firewall logs, converts them to common field names, and sends them to the SIEM. The raw packets stay on the sensor.

4. **Analysis.** Elasticsearch indexes the events. Detection rules run on them: flow analytics (volume, duration, beaconing), signatures and IoC matches, and DNS analytics.

5. **Security alerts.** Each rule match creates an alert with a severity, the hosts involved and the rule that fired.

6. **Triage.** The analyst reviews the alert on the Kibana dashboard, looks at the same host and time window, and decides whether it is a false positive (close it and note a tuning change) or suspicious (investigate).

7. **Investigation.** The analyst pivots from the alert to the host's flows to see all its connections, to the DNS log to find which domains were looked up, and finally pulls the relevant packets from the packet store to confirm what was transferred.

Response and feedback. A confirmed incident is recorded as a case and handled in four steps: contain (block the IP addresses and domains at the firewall, disconnect the workstation), eradicate and recover (clean or reimage the host, reset the user's password), and lessons learned. New IoCs and tuned rules

13 Conclusion

This project analysed two data sets, a lab with HTTP and SYN floods and a public capture of an infected Windows workstation, and took both through the complete security-monitoring process (Table 13.1).

Table 13.1: The security-monitoring process applied in this project.

Stage	**What the project did**
Network data	Captured lab traffic; a 53-minute malware capture, hashed before use
Collection	Zeek and Suricata logs loaded into Elasticsearch and Kibana; probe, resource and nginx data for the lab
Analysis	Packet timing, flow and request-level analysis, security dashboard
Security indicator	Request floods and half-open handshakes; bare-IP polling every 5.3 s, self-signed TLS on port 2917
Investigation	Host DESKTOP-L8C5GSJ and user shutchenson identified; infection chain reconstructed
Detection	Indicators turned into rules; tuning cut alerts from 619 to 18
Decision	Compromise confirmed with high confidence; floods caused degradation, not compromise
Response	Isolate and reimage the host, block C2 infrastructure, feed new rules back

Three lessons stand out. Suspicious behaviour was recognised by combinations of features judged against the host's role, not by a single threshold. Every finding needed a second, independent source before it was accepted. And indicators were short-lived, because the C2 servers were set up hours before use, so behavioural rules such as request-level polling detection gave the durable results.

Network security monitoring is therefore not the collection of packets and logs. Its value comes from turning that data into reliable security evidence that supports detection, investigation and incident response.

14 References

1. NIST SP 800-92, *Guide to Computer Security Log Management*. https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-92.pdf

2. NIST SP 800-61 Rev. 2, *Computer Security Incident Handling Guide*. https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-61r2.pdf

3. NIST SP 800-137, *Information Security Continuous Monitoring (ISCM)*. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-137.pdf

4. Zeek documentation, *conn.log*. https://docs.zeek.org/en/master/reference/logs/conn.html

5. Zeek project. https://zeek.org/

6. M. Tuexen et al., "PCAP Now Generic (pcapng) Capture File Format," IETF Internet-Draft draft-ietf-opsawg-pcapng. https://datatracker.ietf.org/doc/draft-ietf-opsawg-pcapng/

7. Wireshark Foundation, "Display Filter Reference." https://www.wireshark.org/docs/dfref/

8. Zeek Project, "dns.log." https://docs.zeek.org/en/lts/logs/dns.html

9. P. Mockapetris, "Domain Names — Implementation and Specification," RFC 1035, 1987.

10. C. Contavalli et al., "Client Subnet in DNS Queries," RFC 7871, 2016.

11. Zeek Project, "http.log" and "ssl.log." https://docs.zeek.org/en/lts/logs/

12. R. Fielding, M. Nottingham and J. Reschke, "HTTP Semantics," RFC 9110, 2022.

13. E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, 2018.

14. S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," ACM Transactions on Information and System Security, vol. 3, no. 3, pp. 186–205, 2000.

15. R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symposium on Security and Privacy, 2010, pp. 305–316.

16. V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, pp. 1–58, 2009.

17. K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST Special Publication 800-94, 2007.

18. MITRE, "ATT&CK Enterprise Matrix." https://attack.mitre.org/

A Laboratory Details

A.1 Generated-Traffic Lab

Table A.1: Components of the generated-traffic lab.

Component	Address	Role
Target	172.28.0.10	nginx web server in front of a Python application
Attacker	172.28.0.20	Generates the HTTP flood and the SYN flood
Benign probe	172.28.0.30	Requests the page continuously to measure availability and latency
Sniffer	Shares the target's network namespace	Captures all traffic on the target's interface

nginx served `/heartbeat` as a static file and forwarded `/` and `/index.html` to a local Python application. Each page request read and sorted a 5,000-row catalogue before responding, giving a repeatable workload. A local health check ran every two seconds; it appeared in the nginx logs but not in the capture.

The target was limited to half a CPU core and 64 MiB of memory, inside a virtual machine with two vCPUs, about 3.7 GiB of memory and no swap. The other containers had no resource limits. SYN cookies were disabled on the target.

Table A.2: Tools used in the generated-traffic lab.

Tool	Purpose
nginx and Python	Web server and application workload on the target
hey	HTTP flood generation
ZMap	SYN flood generation
curl	Benign probe requests and latency measurement
tcpdump	Packet capture on the target's interface

A.2 Real-Traffic Lab

The Docker Compose stack ran on a single host with four CPUs and 6 GB of memory.

Table A.3: Real-traffic dataset.

Item	Value
File	`2025-01-22-traffic-analysis-exercise.pcap`
Size	26,759,675 bytes; 39,427 packets; 19:44:56 to 20:38:18 UTC
SHA-256	`e59db1c07c6fdefafa0abdbca03248c341cdc36c09c34753204d3162802a3586`
Scenario	A user in an Active Directory network searched for Google Authenticator, downloaded a fake installer and the host then contacted attacker infrastructure
Network	10.1.17.0/24, domain bluemoontuesday.com, domain controller 10.1.17.2

Table A.4: Components of the real-traffic lab.

Component	Role
Zeek	Converts packets into connection, DNS, HTTP, TLS, certificate and file logs
Suricata	Matches packets against ET Open and the group's local rules
Filebeat	Ships Zeek and Suricata logs into Elasticsearch in a common schema
Elasticsearch and Kibana	Store and search events; host the security dashboard
Flow-statistics script	Scores beaconing and HTTP request rhythm from the Zeek logs
tshark, Wireshark and Python	Packet inspection and the independent flow table